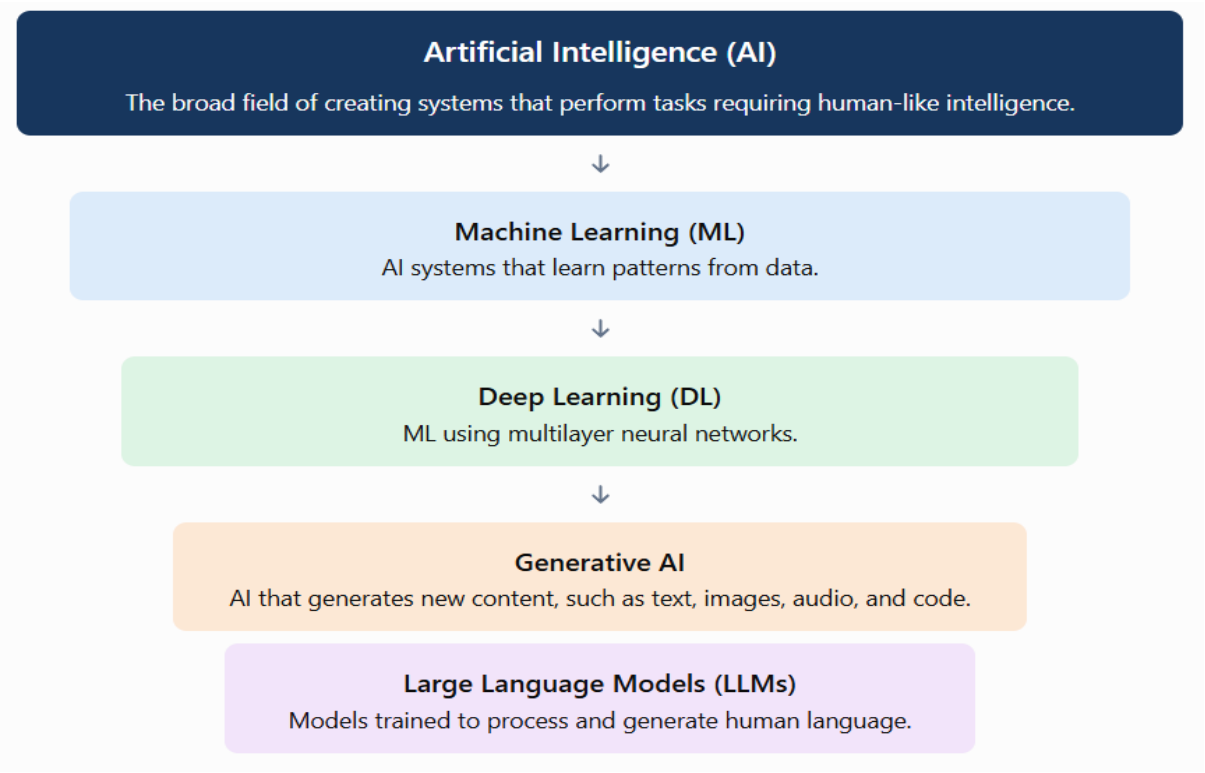
MISSION 1: Build the AI and ML Landscape

Research Question

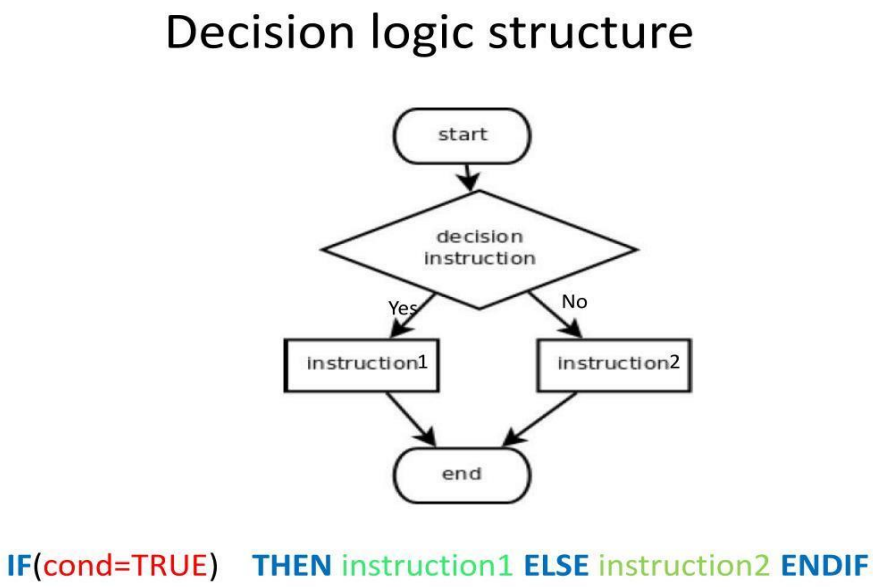
What is Artificial Intelligence (AI), and where exactly does Machine Learning (ML) fit?

1. Concept Map: AI and ML Landscape



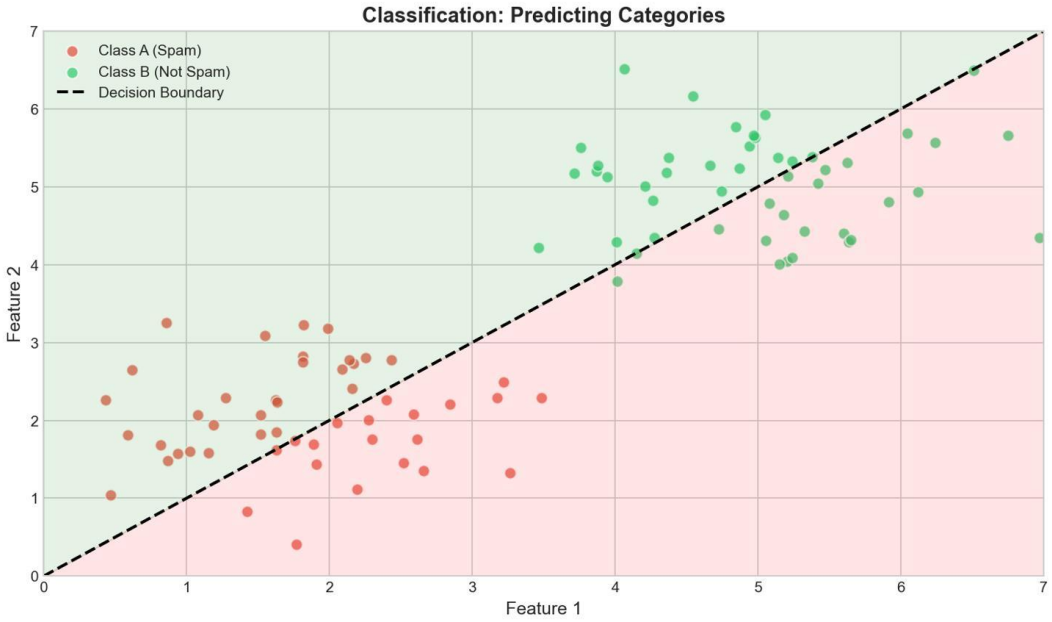
Note: This is a simplified hierarchy. Generative AI also includes methods outside deep learning, and not all AI systems use ML.

2. Five Key Distinctions



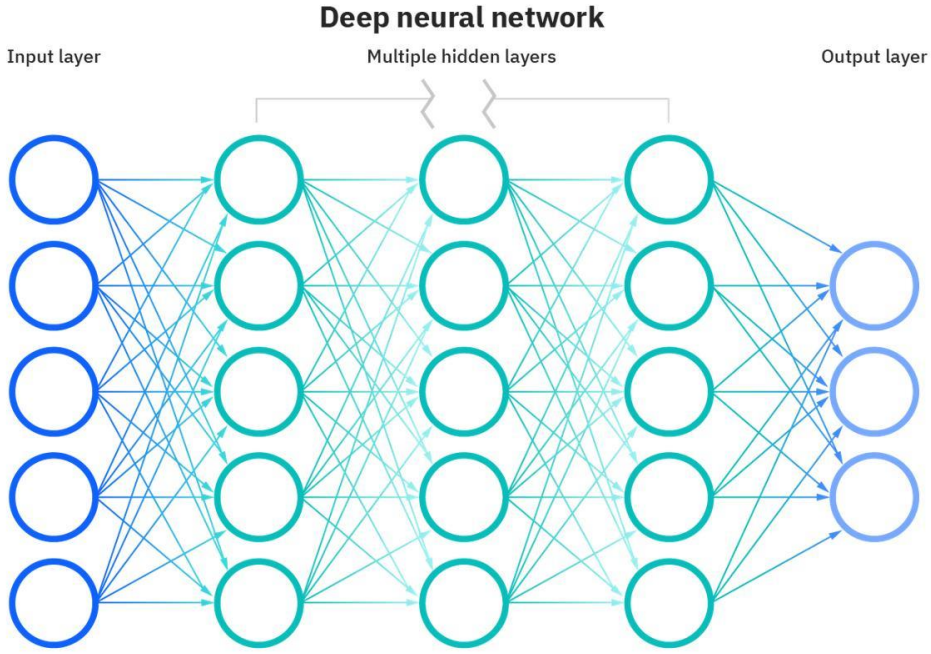
1. AI vs. Conventional Rule-Based Programming

Conventional programming follows explicit instructions written by humans. AI is the broader field of building systems that perform tasks such as reasoning, perception, prediction, and language understanding. Some AI systems use rules rather than learning from data.



2. Machine Learning vs. Rule-Based Programming

Rule-based programs rely on predefined logic. ML algorithms learn patterns from examples and use those patterns to make predictions or decisions on new data.



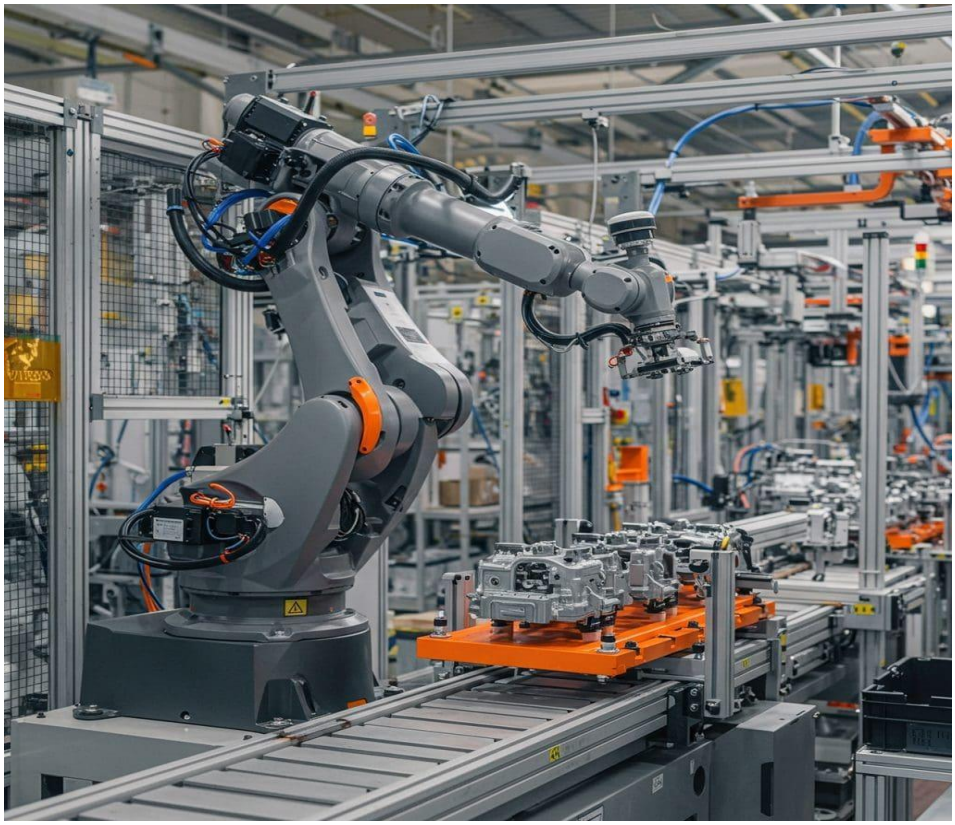
3. Machine Learning vs. Deep Learning

ML includes many learning techniques, such as decision trees and regression. Deep learning is a subset of ML that uses neural networks with multiple layers to learn complex patterns.



4. AI vs. Generative AI and LLMs

Generative AI creates new content. LLMs are a type of AI model specialized in language tasks, including writing, summarizing, answering questions, and generating code. Modern LLMs generally use deep learning.



5. Automation vs. AI

Automation executes tasks with limited human intervention. It can use simple scripts, timers, or fixed rules without intelligence or learning. AI can make automation more adaptive, but automation does not necessarily require AI.

3. Why Automation Is Not Necessarily AI

Automation and AI solve different problems.

- Automation focuses on performing tasks automatically.
- AI focuses on performing tasks associated with intelligence.
- ML focuses on learning patterns from data.

For example, a washing machine running a fixed wash cycle is automation, not necessarily AI. A washing machine that learns from sensor data to optimize washing conditions may incorporate ML.

Similarly, not all AI uses ML: a chess program that selects moves using explicitly programmed rules and search algorithms can be an AI system without learning from data.

4. What Is Explicitly Programmed vs. Learned from Data?

Aspect	**Explicitly programmed**	**Learned from data**
Instructions	Written by developers	Patterns inferred during training
Behavior	Follows predefined rules	Depends on the trained model
Adaptation	Requires changing the rules	May improve through retraining
Example	If temperature exceeds a limit, switch on a fan	Predict room temperature using historical sensor data

Many real-world systems combine both approaches. For example, a spam filter may use an ML model to classify emails while explicit rules block known dangerous senders.

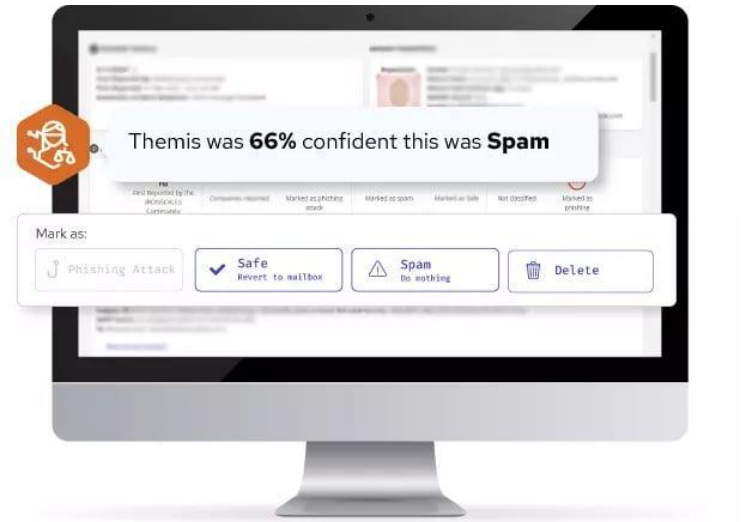
5. Three Examples Classified With Reasons



Example 1: Traffic light with a fixed timer

Rule-based automation

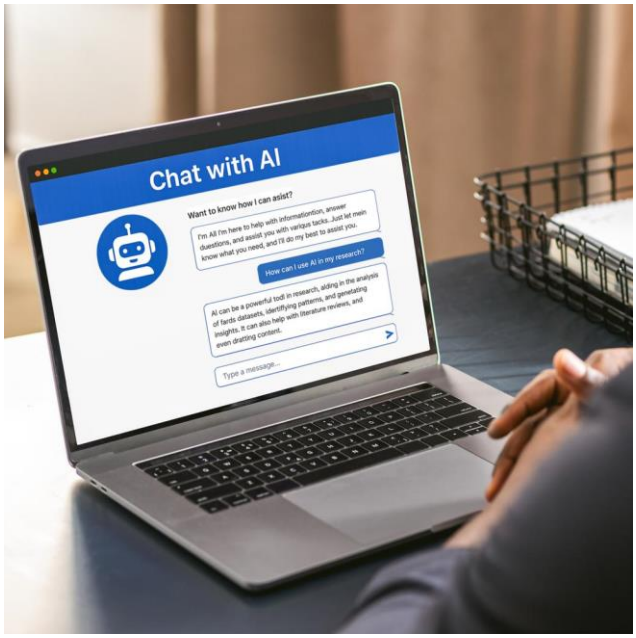
Reason: It switches lights according to predefined timings. It does not need to learn from data or make intelligent predictions.



Example 2: Email spam detection

Machine Learning

Reason: An ML model learns patterns from examples of spam and legitimate emails, then predicts the category of new messages. It is ML and therefore a form of AI.



Example 3: ChatGPT generating an essay

Generative AI / LLM

Reason: An LLM uses deep learning to generate text based on its training and the conversation context. It is generative AI, a form of AI built using ML.

Conclusion

Artificial Intelligence is the broadest concept. Machine Learning is one approach within AI, Deep Learning is a subset of ML, and modern Generative AI and LLMs commonly rely on deep learning. Conventional programming and automation can operate without AI or ML, although they can also be combined with them.

MISSION 2: Understand Data — What Does an ML System Learn From?

Research Question

What does it mean to learn a pattern from data?

Learning a pattern from data means identifying relationships, trends, or repeated behaviors in examples and using them to make predictions or decisions about new, unseen examples.

For example, an ML model can learn from historical chip-design measurements to predict whether a new chip design will pass or fail its timing requirements.

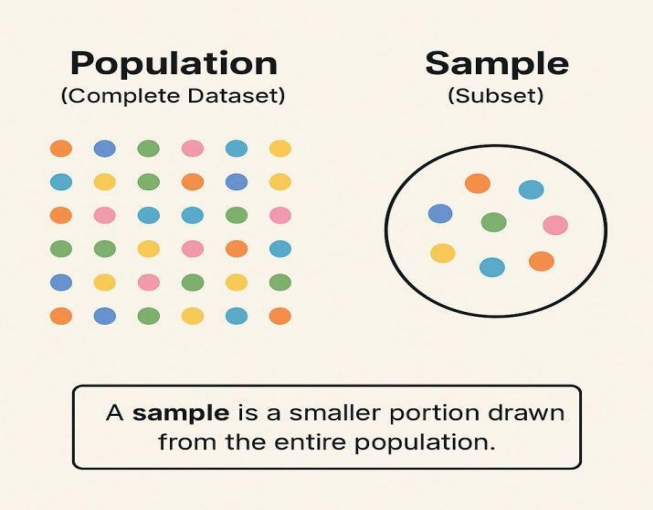
1. Understanding the Basic Data Concepts

	A	B	C	D	E
1	**sepal length**	**sepal width**	**petal length**	**petal width**	**class**
2	5.1	3.5	1.4	0.2	Iris-setosa
3	4.9	3	1.4	0.2	Iris-setosa
4	4.7	3.2	1.3	0.2	Iris-setosa
5	4.6	3.1	1.5	0.2	Iris-setosa
6	5	3.6	1.4	0.2	Iris-setosa
7	5.4	3.9	1.7	0.4	Iris-setosa
8	4.6	3.4	1.4	0.3	Iris-setosa
9	5	3.4	1.5	0.2	Iris-setosa

1. Dataset

A collection of related data used for analysis or machine learning.

Example: A table containing chip area, power consumption, delay, and timing results for 100 chip designs.



2. Sample

One selected example or, in statistics, a subset of a larger population.

Example: One chip design selected from a dataset of 100 designs.

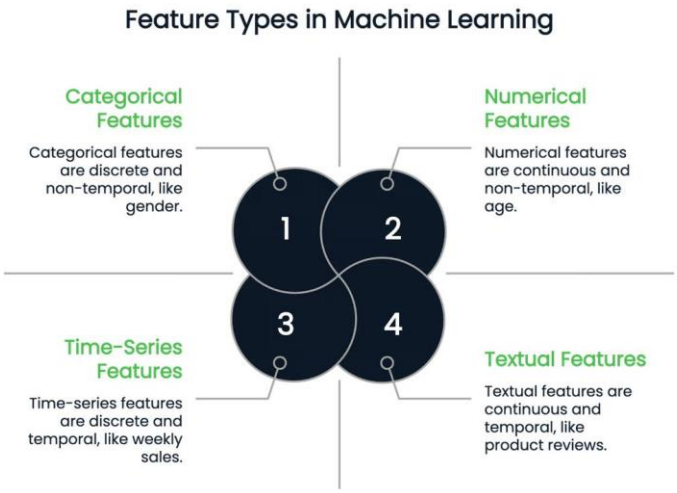
1D Tabular Data

	ID	Severity	Gender	Income	Age
	20	O	Male	5435	55
	21	BC	Female	8541	42
	22	KA	Female	6582	23
Sample	**23**	**KA**	**Male**	**4582**	**30**
	24	O	Female	8342	28
	25	O	Male	8642	56
	26	BC	Male	6575	37

3. Observation

One recorded case or instance, usually represented by a row in a dataset.

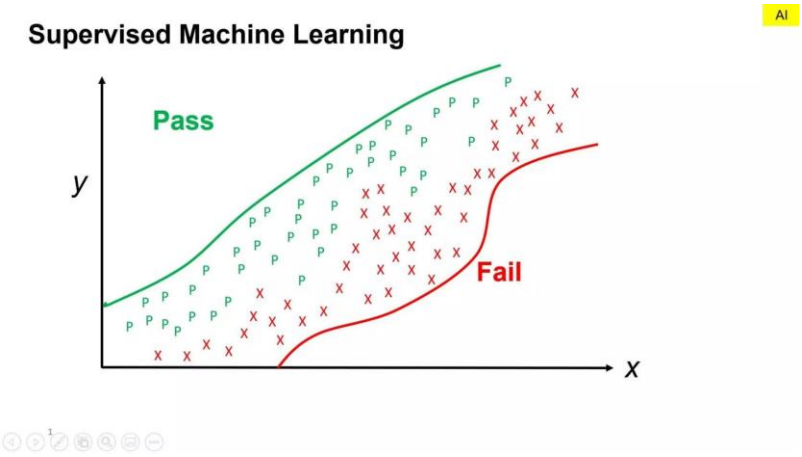
Example: The measurements recorded for one particular chip design.



4. Feature

An input variable used by a model to learn patterns or make predictions.

Examples: Chip area, power consumption, operating frequency, and design type.

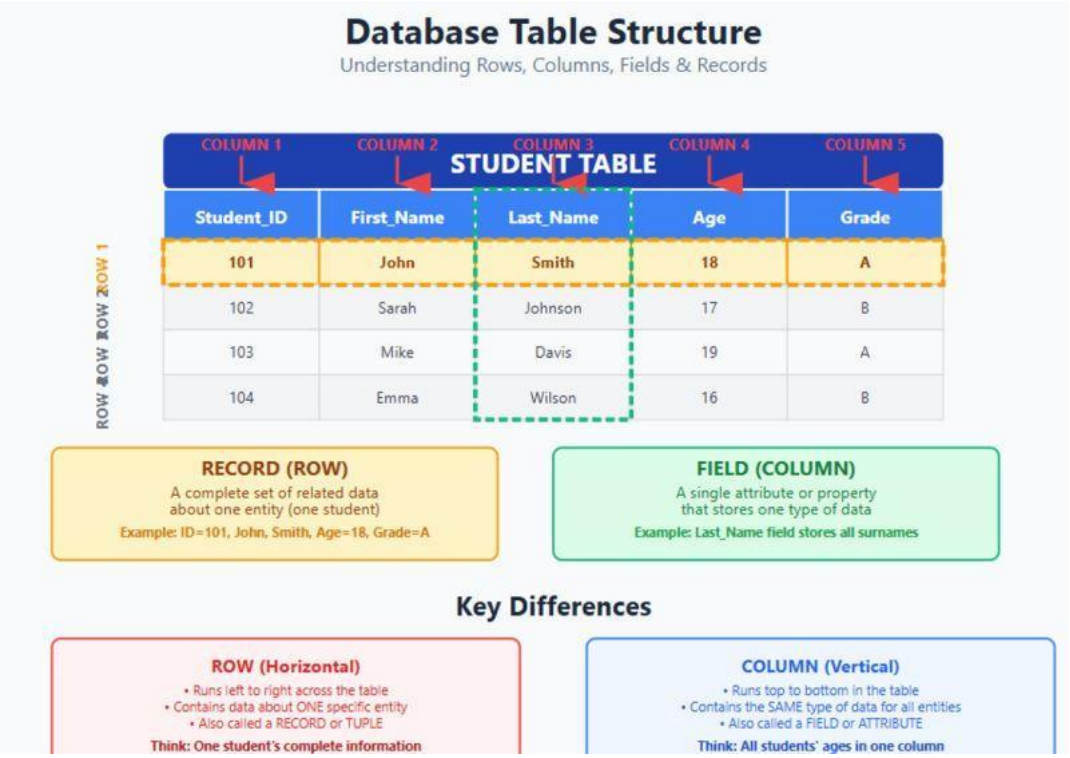


5. Label and Target

The known answer associated with an observation in supervised learning. The target is the outcome the model is trained to predict.

Example: Pass or Fail for a chip's timing verification.

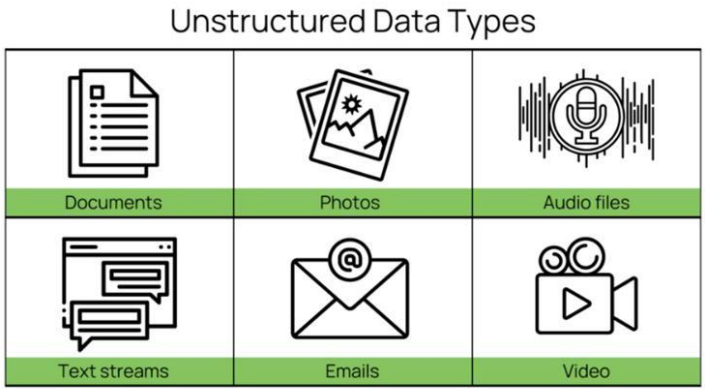
2. Structured vs. Unstructured Data



Structured data

Data organized into a predefined format, such as rows and columns.

Examples: spreadsheets, database tables, chip measurements, customer records.



Unstructured data

Data that does not naturally fit into a fixed tabular schema.

Examples: photographs, audio recordings, free-text documents, and videos.

Structured data is often easier to analyze with conventional tabular ML methods. Unstructured data can also be used for ML, but it often requires specialized models or preprocessing.

3. Numerical vs. Categorical Features

Type	**Meaning**	**Examples**
Numerical	Values representing quantities or measurements	Area = 120 mm², delay = 8 ns, power = 45 W
Categorical	Values representing groups or classes	Process = 5 nm, design type = CPU, status = Pass

Numerical features may be continuous, such as voltage, or discrete, such as the number of logic gates. Categorical features may be nominal, such as chip type, or ordinal, such as low, medium, and high.

4. Labelled vs. Unlabelled Data

Labelled data — Supervised learning

The inputs and the correct outcomes are provided.

Area (mm²)	**Delay (ns)**	**Timing result**
80	6.2	Pass
100	8.1	Pass
120	11.4	Fail

The model learns the relationship between chip measurements and timing results.

Unlabelled data — Unsupervised learning

The inputs are available, but the correct outcome labels are not.

Area (mm²)	Delay (ns)
80	6.2
100	8.1
120	11.4

The model could discover groups of similar chip designs or identify unusual measurements without being told which designs pass or fail.

5. Data Collection and Data Quality

Before training an ML model, the data must be collected and prepared carefully.

Data collection

Gather relevant measurements from reliable sources, such as simulations, sensors, experiments, or public datasets.

Data cleaning

Remove duplicates, correct formatting errors, and handle invalid values.

Missing values

Some observations may lack measurements. Depending on the context, these values may be imputed, explicitly marked, or the affected records removed.

Noisy data

Measurement errors or random fluctuations can hide genuine patterns. Investigate suspicious values rather than automatically deleting every outlier.

Inconsistent labels

If similar chip designs are incorrectly marked as both Pass and Fail, the model may learn contradictory relationships. Define consistent labeling rules and verify results.

Why quantity alone does not guarantee quality

A large dataset can still be unreliable if it contains incorrect measurements, missing information, duplicated records, biased examples, or inconsistent labels.

For example, 10,000 chip-design records with incorrect timing results may train a worse model than 500 carefully verified records. Both quantity and quality matter, as do representative coverage and correct evaluation.

6. Deliverable: A Small Synthetic Dataset

The following six-row dataset is illustrative and uses synthetic chip-design measurements. The target is whether a design passes a timing limit of 10 ns.

Design ID	Area (mm²)	Power (W)	Delay (ns)	Design type	Target: Timing result
C01	80	30	6.2	CPU	Pass
C02	90	34	7.5	CPU	Pass
C03	100	38	8.1	GPU	Pass
C04	110	42	9.6	GPU	Pass
C05	120	45	11.4	CPU	Fail
C06	130	50	12.1	GPU	Fail

Illustrative data only; these values do not represent measured results from a real chip.

Identify the data elements:

- Dataset: the complete six-row table.
- Observation: one row, such as design C03.
- Features: area, power, delay, and design type.
- Target: timing result (Pass or Fail).
- Numerical features: area, power, and delay.
- Categorical feature: design type.
- Labelled data: each design has a known timing result.

In this example, a supervised ML classifier could learn that designs with larger delays tend to fail the timing limit. It would need additional varied examples to learn a reliable general pattern.

7. Two Ways Poor Data Quality Can Affect a Model

1. Incorrect or inconsistent labels

If design C05 has a delay of 11.4 ns but is mistakenly labelled Pass, the model receives contradictory training information. It may learn an incorrect decision boundary and misclassify future designs.

2. Missing or noisy measurements

If delay values are missing or recorded inaccurately, the model may fail to identify the relationship between design measurements and timing outcomes. Its predictions can become less reliable, particularly on new designs.

Conclusion

An ML system learns by finding useful patterns in its training data. Features provide the inputs, and labels provide known answers in supervised learning. Data collection, cleaning, consistency, and

representativeness strongly influence how well the model learns and how reliably it performs on unseen data.

MISSION 3: Understand the Main Machine Learning Approaches

Research Question

How does a system learn when it has labels, no labels, or feedback from actions?

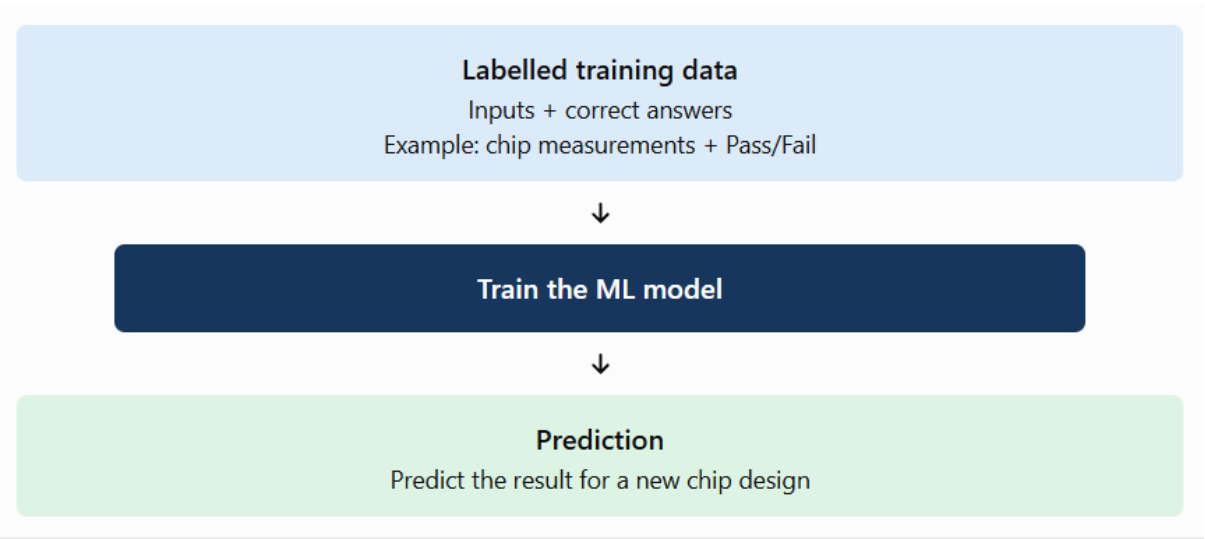
Machine learning (ML) systems learn in different ways depending on the information available during training. The three main approaches are supervised learning, unsupervised learning, and reinforcement learning. Self-supervised learning is another important approach in which a system creates training signals from the data itself.

1. Comparison Table of ML Approaches

Aspect	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Training signal	Labelled examples	Patterns and structure in data	Rewards or penalties from actions
How it learns	Maps inputs to known outputs	Finds hidden patterns or groups	Learns actions that maximize long-term reward
What it produces	Predictions or classifications	Clusters, representations, or anomalies	A policy for choosing actions
Example	Predicting whether a chip passes timing	Grouping similar chip designs	Teaching a robot to move efficiently
One limitation	Needs reliable labelled data	Discovered patterns may be difficult to interpret	Feedback can be expensive or slow to obtain

2. Simple Diagram of Each Learning Process

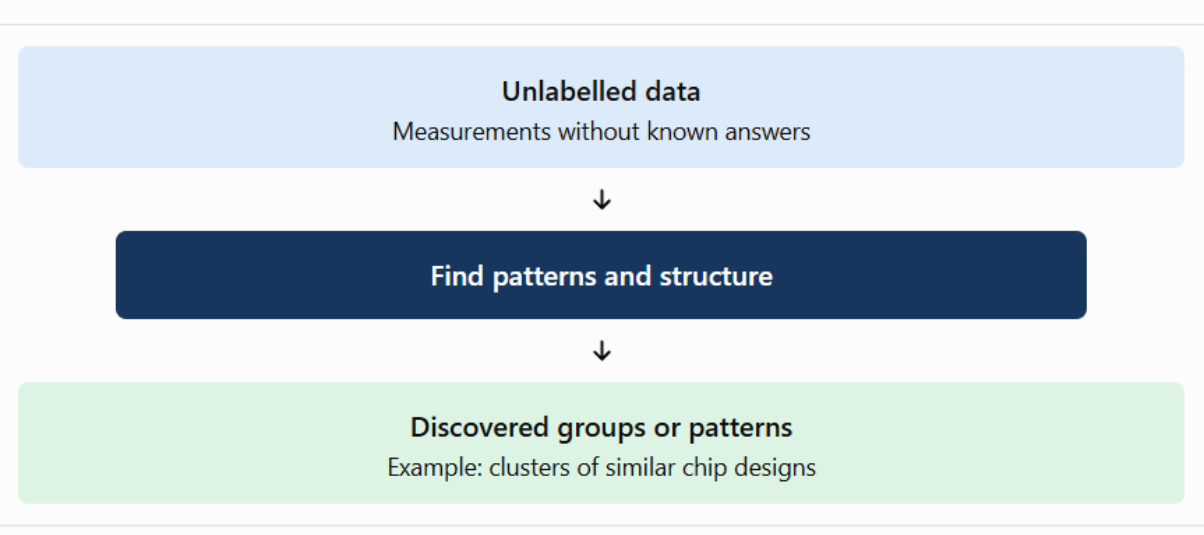
A. Supervised Learning



How it works: The model compares its predictions with known answers and adjusts its parameters to reduce prediction errors.

- Training signal: Correct labels.
- Output: A model that predicts values or categories.
- Example: Predicting whether a chip design will pass its timing requirement.
- Limitation: Label creation may require expensive experiments or expert review.

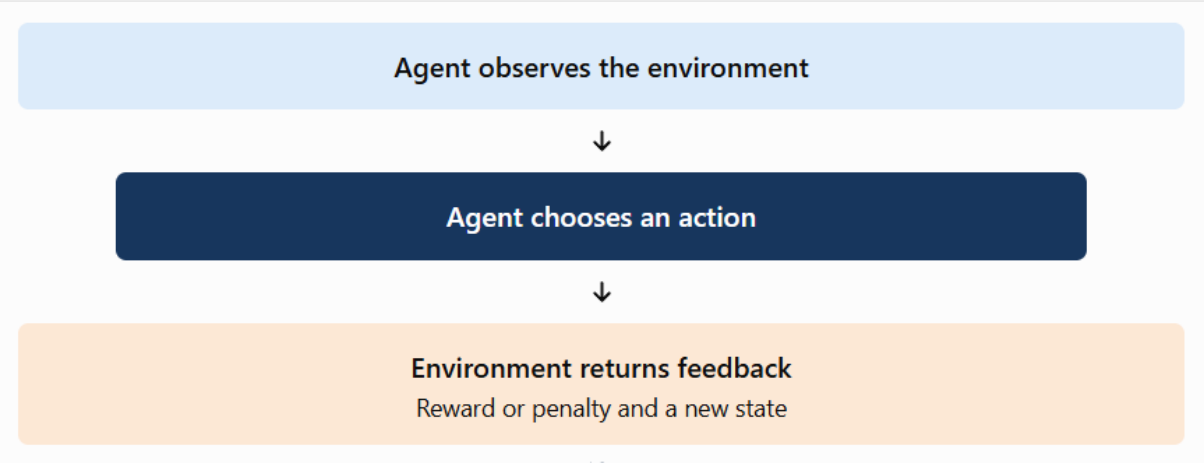
B. Unsupervised Learning



How it works: The algorithm identifies similarities, differences, or unusual observations without relying on predefined target labels.

- Training signal: Structure in the input data.
- Output: Clusters, anomaly scores, or lower-dimensional representations.
- Example: Grouping chip designs by similar area, power, and delay characteristics.
- Limitation: A discovered cluster may not correspond to a meaningful real-world category.

C. Reinforcement Learning



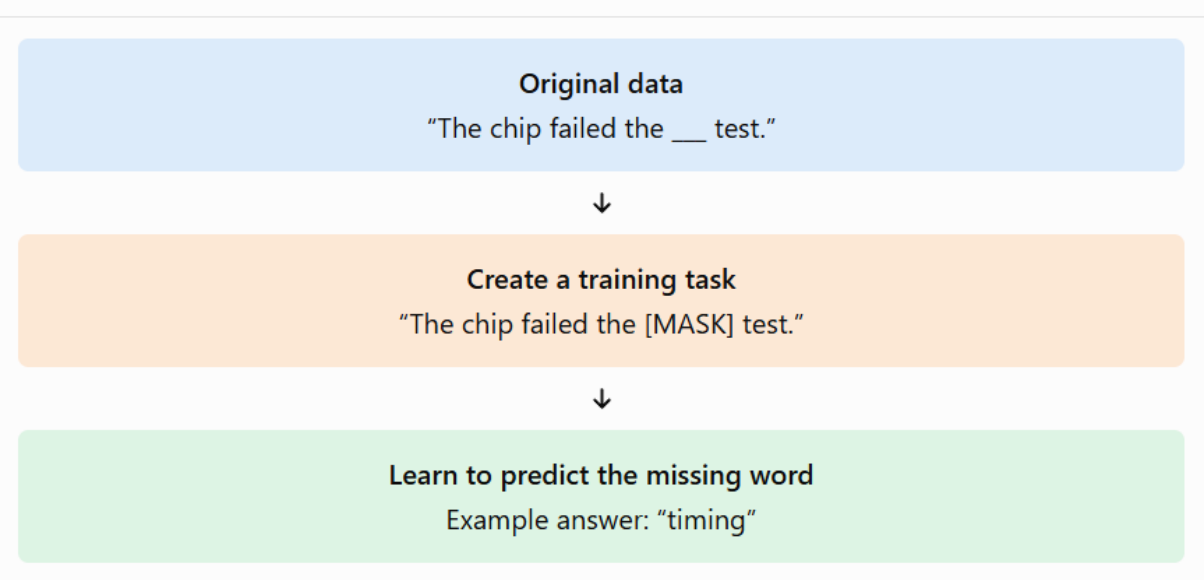
How it works: An agent interacts with an environment, receives rewards or penalties, and learns which actions produce better long-term outcomes.

- Training signal: Rewards and environmental feedback.
- Output: A policy that selects actions in different situations.
- Example: Teaching a robot to navigate a room while avoiding obstacles.
- Limitation: Learning can require many interactions, and poorly designed rewards can encourage undesirable behavior.

3. Self-Supervised Learning: Learning From the Data Itself

Self-supervised learning is an approach where training targets are created automatically from the input data rather than supplied as human-provided labels.

For example, a model can learn from a sentence by hiding one word and training itself to predict the missing word.



The example is illustrative: the model learns by predicting a hidden part of the data. It does not need a person to label every sentence.

Self-supervised vs. unsupervised learning

Aspect	Unsupervised learning	Self-supervised learning
Main idea	Discover structure in data	Generate a learning task from the data
Training target	Often no explicit target label	Automatically generated target
Example	Group similar chip designs	Predict a masked word or missing data segment
Common use	Clustering and anomaly detection	Learning useful representations from large datasets

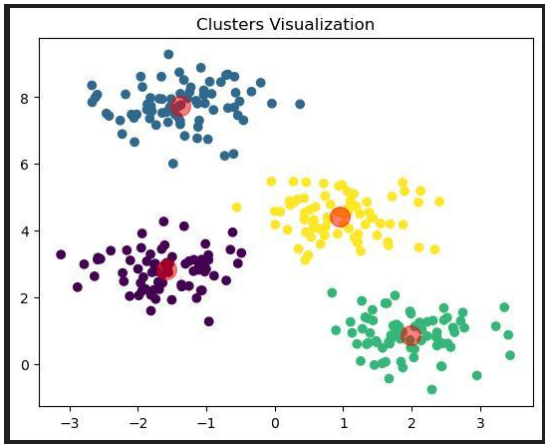
Why they are not always interchangeable: Both can use unlabelled data, but self-supervised learning explicitly constructs a prediction task or training signal. It is often treated as a distinct learning paradigm because of this training mechanism, even though it is also commonly discussed within the broader family of unsupervised methods.

4. When Should Each Approach Be Used?



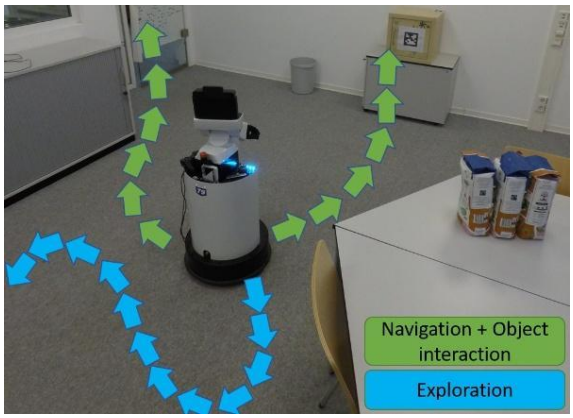
Use supervised learning when answers are known.

Example: You have historical chip measurements and verified Pass/Fail results and want to predict timing success for new designs.



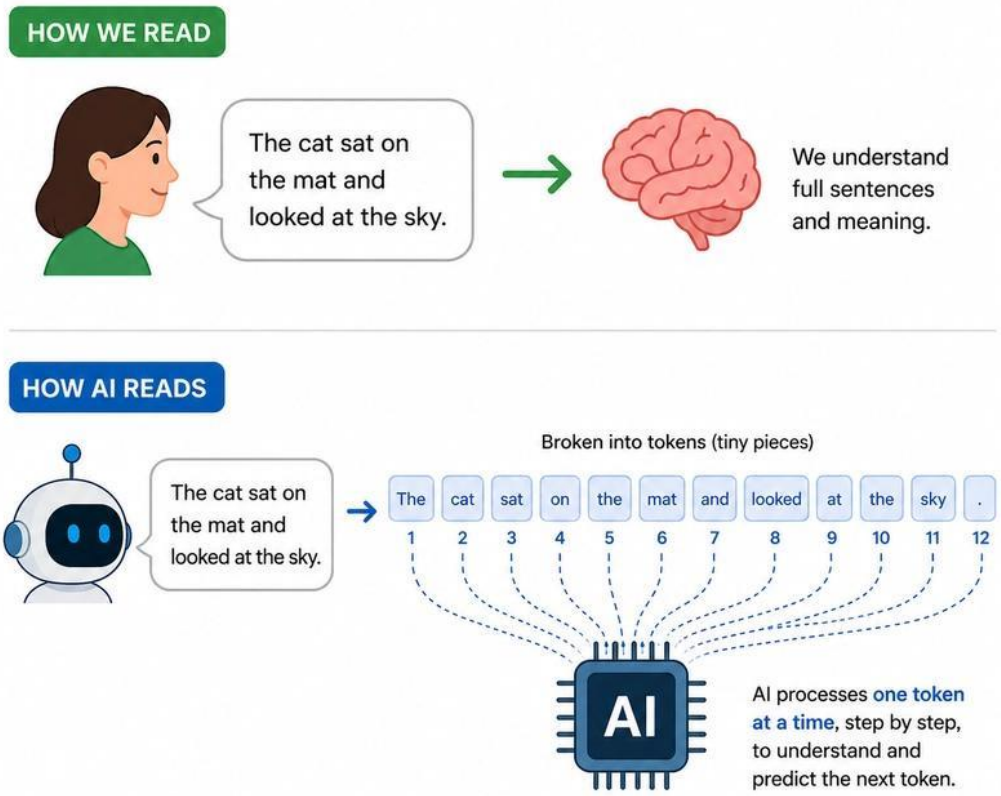
Use unsupervised learning when exploring data.

Example: You want to discover groups of chip designs with similar performance without existing categories.



Use reinforcement learning when actions affect outcomes.

Example: An agent tests different design-optimization actions and receives rewards based on timing, area, and power constraints.



Use self-supervised learning when large amounts of raw data are available.

Example: A model learns patterns from technical documents by predicting masked words or the next token, before being adapted to a particular task.

Conclusion

The main difference between ML approaches is the training signal each one uses:

- Supervised learning learns from labelled examples.
- Unsupervised learning discovers structure in unlabelled data.
- Reinforcement learning learns from the consequences of actions.
- Self-supervised learning generates training targets from the data itself.

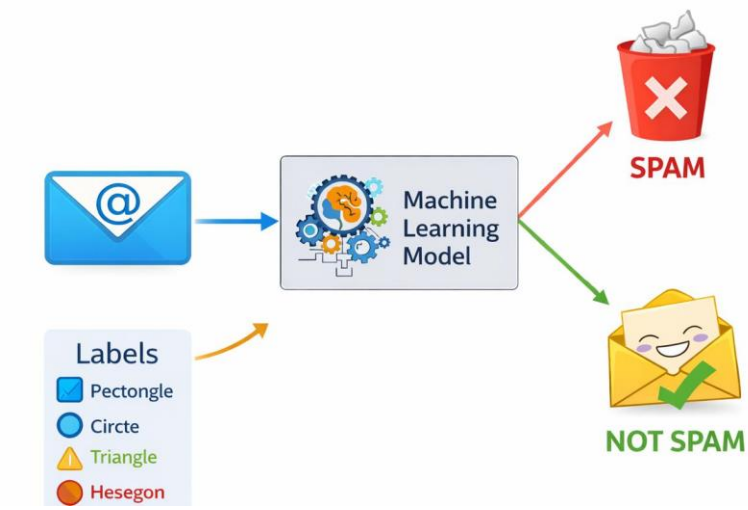
MISSION 4: Understand Machine Learning Task Types

Research Question

What kinds of problems can machine learning solve?

Machine learning can solve different types of problems depending on the desired output. It can classify items into categories, predict numerical values, discover groups, detect unusual events, simplify data, or recommend relevant choices.

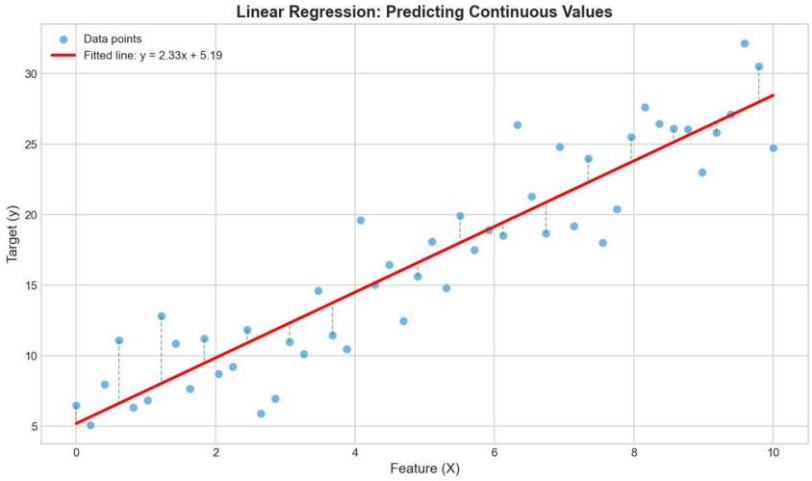
1. Six Main Machine Learning Task Types



1. Classification

Predicts a category or class from input data.

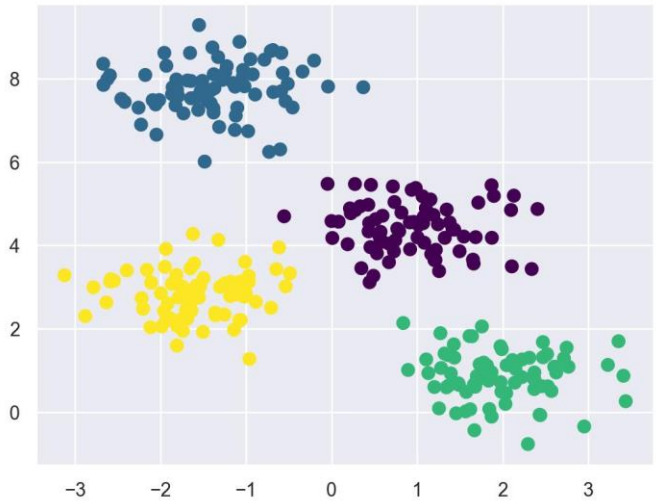
- Input: Chip measurements such as area, power, and delay.
- Output: Pass or Fail.
- Example: Predicting whether a semiconductor chip meets timing requirements.



2. Regression

Predicts a numerical value.

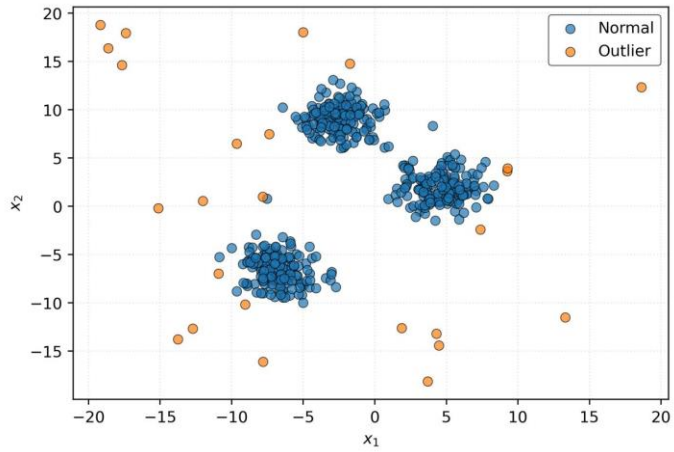
- Input: Chip area, voltage, temperature, and operating frequency.
- Output: Predicted power consumption, such as 42.5 W.
- Example: Estimating the power consumption of a new chip design.



3. Clustering

Groups similar data points without requiring predefined class labels.

- Input: Chip measurements from many designs.
- Output: Groups of similar designs.
- Example: Grouping chips with similar power, area, and performance characteristics.

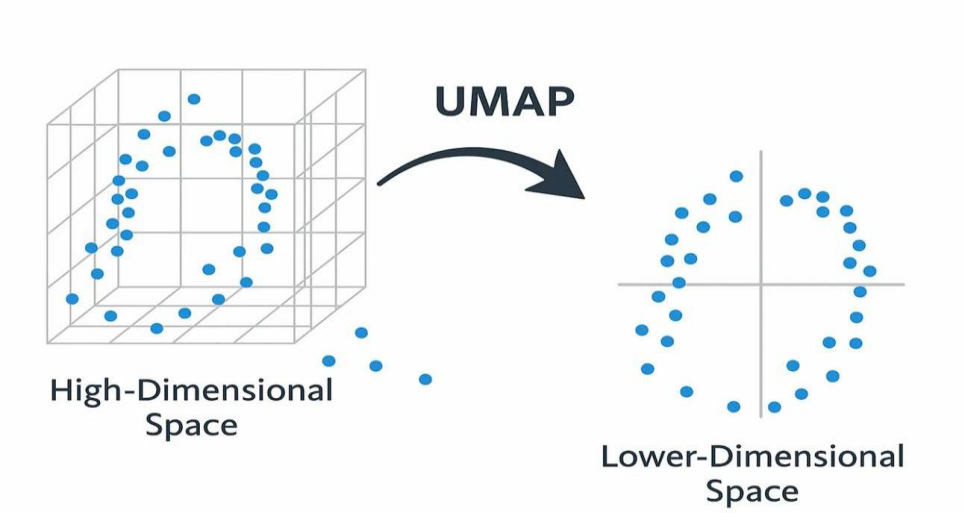


4. Anomaly Detection

Identifies observations that differ significantly from expected patterns.

- Input: Sensor readings or chip test measurements.
- Output: An anomaly score or an alert for a suspicious result.

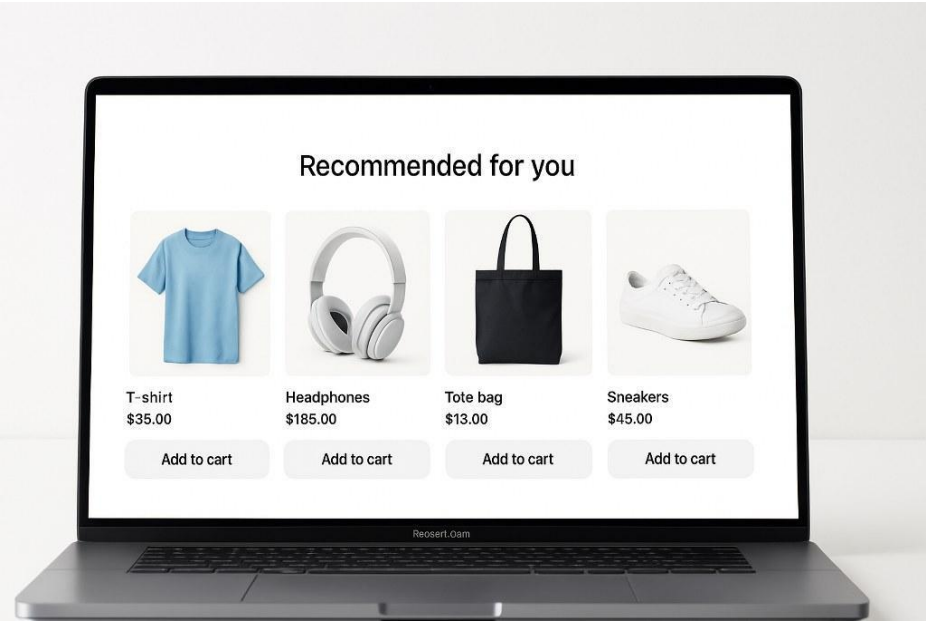
- Example: Detecting an unusual power spike that could indicate a chip manufacturing defect.



5. Dimensionality Reduction

Represents data using fewer variables while preserving important information.

- Input: Hundreds of chip-design measurements.
- Output: A smaller set of features or a low-dimensional representation.
- Example: Reducing hundreds of correlated measurements to two dimensions for visualization.



6. Recommendation and Ranking

Recommends relevant options or orders them by predicted relevance, quality, or usefulness.

- Input: User preferences, design constraints, and historical results.
- Output: Recommended or ranked choices.

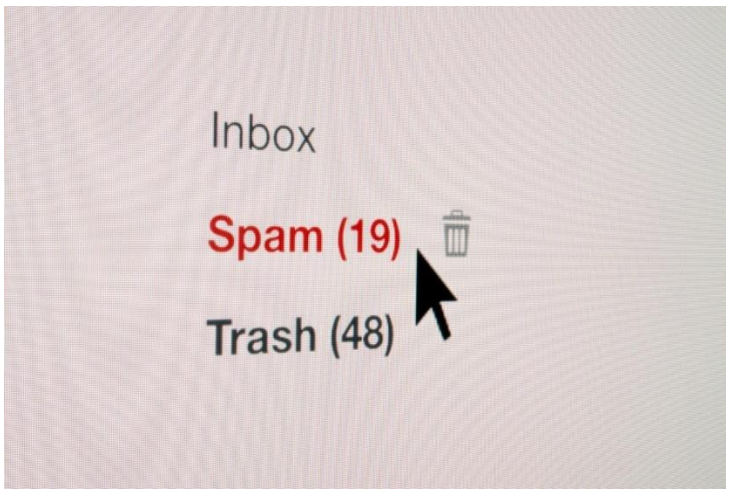
- Example: Ranking chip-design configurations by expected performance and power efficiency.

2. Comparison Table: Inputs and Outputs

Task type	Likely input	Expected output	Example
Classification	Features and labelled examples	Class or category	Chip passes or fails
Regression	Numerical or categorical features	Continuous numerical value	Predict chip power
Clustering	Unlabelled feature vectors	Group assignments	Group similar chip designs
Anomaly detection	Measurements or feature vectors	Anomaly score or flag	Detect abnormal sensor readings
Dimensionality reduction	High-dimensional data	Fewer features or compact representation	Visualize chip data in 2D
Recommendation/ranking	Items, preferences, context	Ranked items or recommendations	Rank design alternatives

3. Classifying Six Real-World Problems

Here are six examples, including four from everyday applications and two additional semiconductor-related examples.



Example 1: Email spam filtering

Classification

Input: Email text, sender details, and message features.

Output: Spam or legitimate.

Reason: The model selects a category.



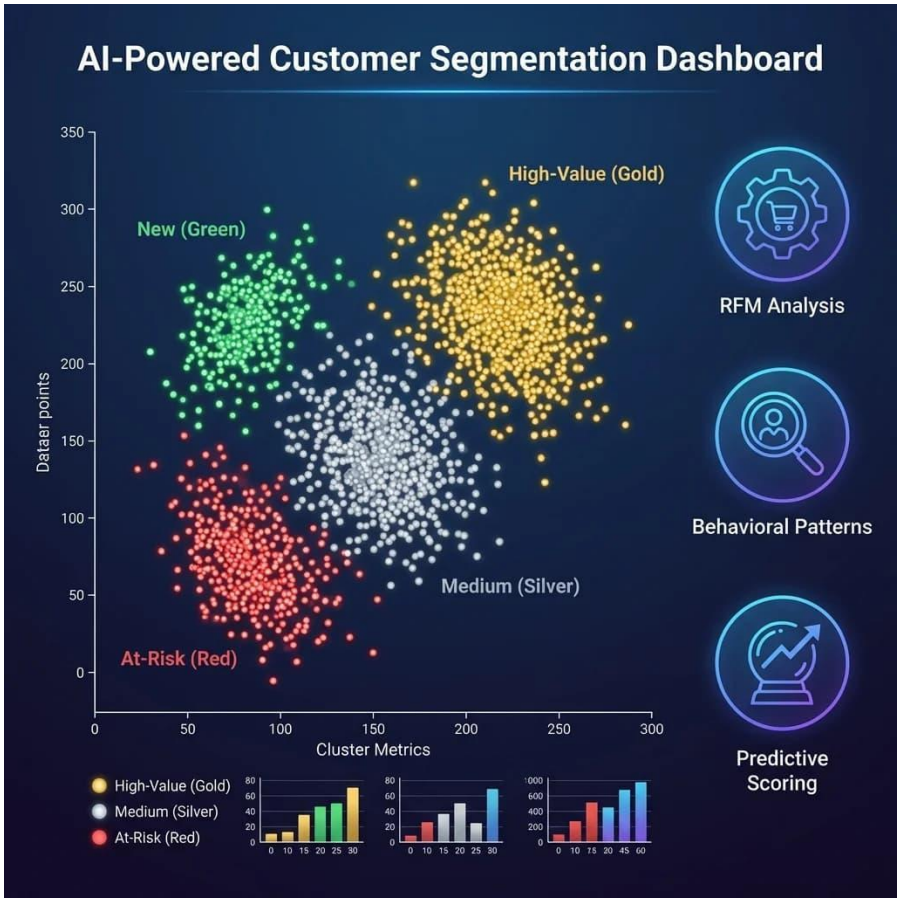
Example 2: House price prediction

Regression

Input: Location, size, age, and number of rooms.

Output: Estimated house price.

Reason: The target is a numerical value.



Example 3: Customer segmentation

Clustering

Input: Purchase history, spending, and browsing behavior.

Output: Groups of customers with similar behavior.

Reason: The groups are discovered from the data.



Example 4: Detecting machine faults

Anomaly detection

Input: Vibration, temperature, and pressure measurements.

Output: An alert or anomaly score.

Reason: The system identifies unusual behavior.



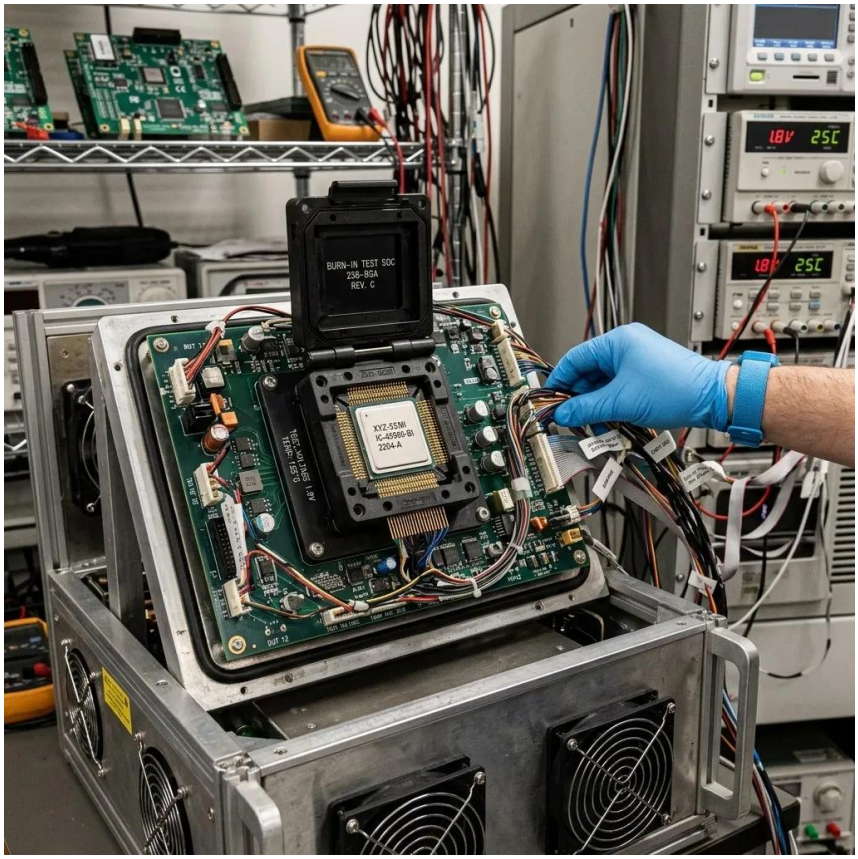
Example 5: Predicting chip timing success

Classification

Input: Historical chip-design measurements, such as delay, area, and frequency.

Output: Timing pass or fail.

Reason: The expected answer is a category.



Example 6: Estimating chip power consumption

Regression

Input: Voltage, frequency, temperature, and design characteristics.

Output: Estimated power consumption in watts.

Reason: The expected answer is a numerical measurement.

4. Task Type vs. Learning Paradigm

These two concepts describe different aspects of machine learning.

- Task type describes what the model is trying to do.
- Learning paradigm describes how the model learns from available information.

Learning paradigm	Typical tasks	Example
Supervised learning	Classification, regression	Learn from chip measurements with known timing results

Unsupervised learning	Clustering, dimensionality reduction	Discover groups in unlabelled chip data
Reinforcement learning	Sequential decision-making and optimization	Learn which design adjustments improve a reward
Self-supervised learning	Representation learning and prediction tasks	Learn patterns from unlabelled technical documents

These are common associations, not strict one-to-one mappings. For example, anomaly detection can be supervised or unsupervised, and dimensionality reduction can use either unsupervised or supervised methods. Recommendation systems can also use supervised learning, collaborative filtering, or reinforcement learning.

5. Why Different Algorithms Can Solve Similar Tasks

Several algorithms can address the same task because they learn patterns in different ways.

Task	**Possible algorithms**
Classification	Logistic regression, decision trees, support vector machines, neural networks
Regression	Linear regression, random forests, gradient boosting
Clustering	K-means, hierarchical clustering, DBSCAN
Anomaly detection	Isolation Forest, One-Class SVM, autoencoders
Dimensionality reduction	PCA, t-SNE, UMAP, autoencoders
Recommendation/ranking	Collaborative filtering, learning-to-rank models, neural recommenders

For instance, a decision tree and a neural network can both classify a chip as Pass or Fail. The best choice depends on factors such as dataset size, accuracy requirements, interpretability, computational cost, and the consequences of incorrect predictions.

Conclusion

Machine learning task types are defined by the problem and the expected output: a class, a number, a group, an anomaly, a compact representation, or a ranked list. The same task can be solved by different algorithms and learning paradigms.

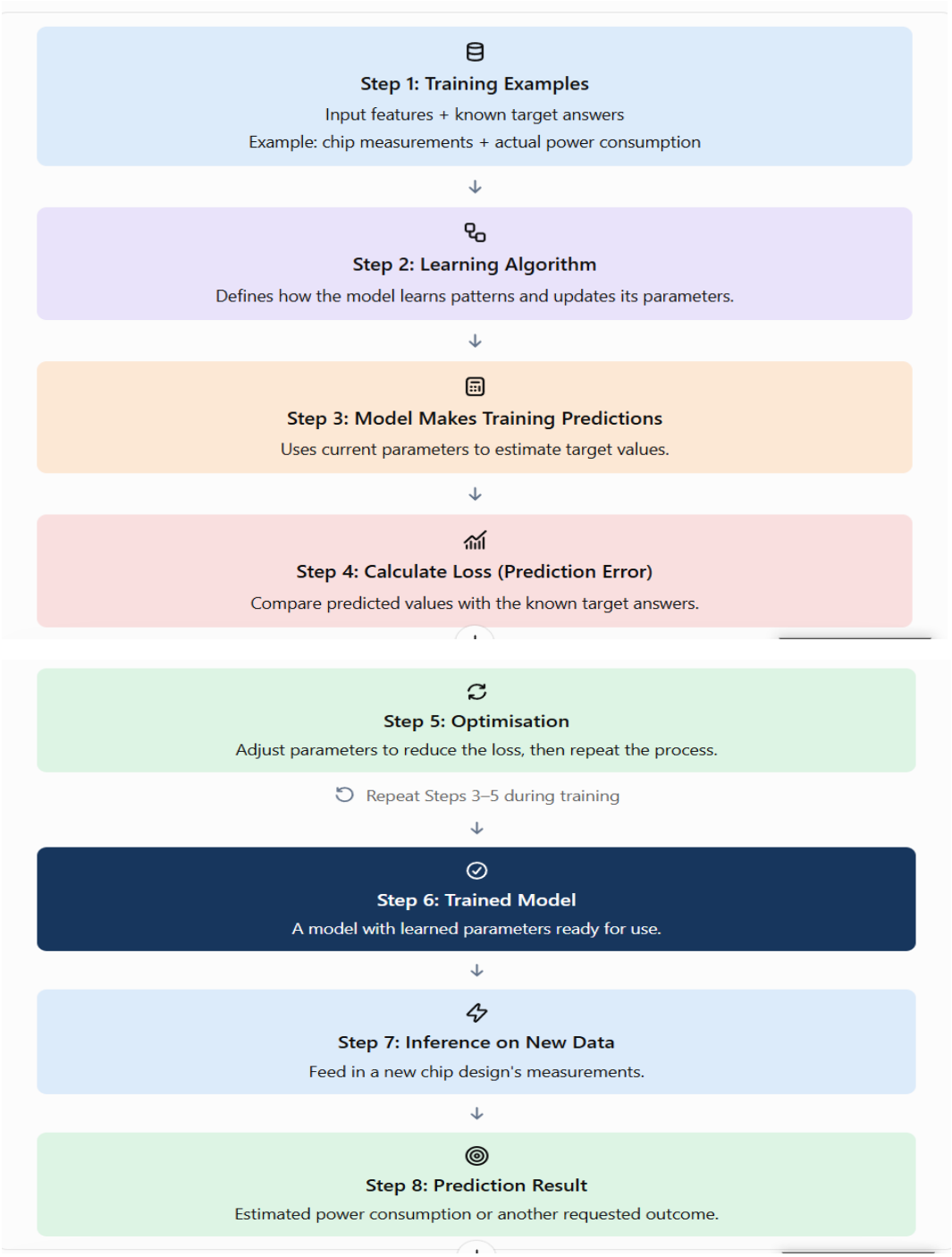
MISSION 5: Understand the Anatomy of an ML Model

Research Question

What happens between feeding data into an ML system and receiving a prediction?

A machine learning system takes examples, uses a learning algorithm to find patterns, adjusts its internal parameters during training, and produces a trained model. That model can then make predictions on new inputs during inference.

1. Labelled Flow Diagram: From Data to Prediction



2. Algorithm vs. Model vs. Learned Parameters

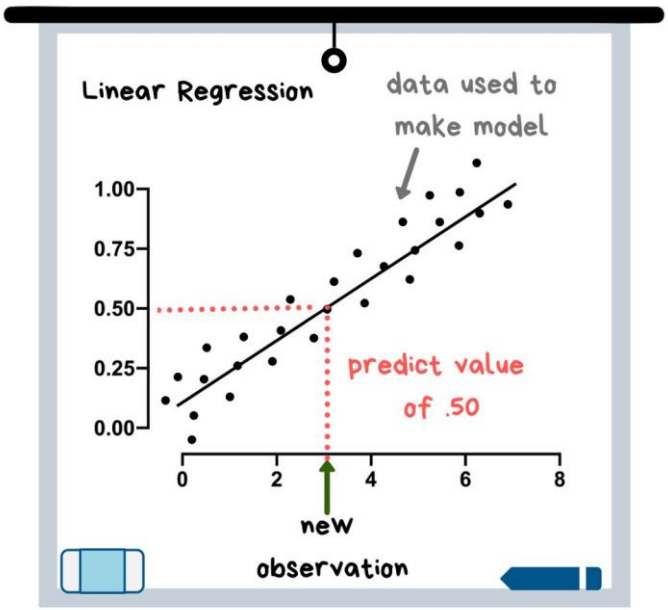
These three terms are related but have different meanings.



Algorithm

The procedure or method used to learn from data.

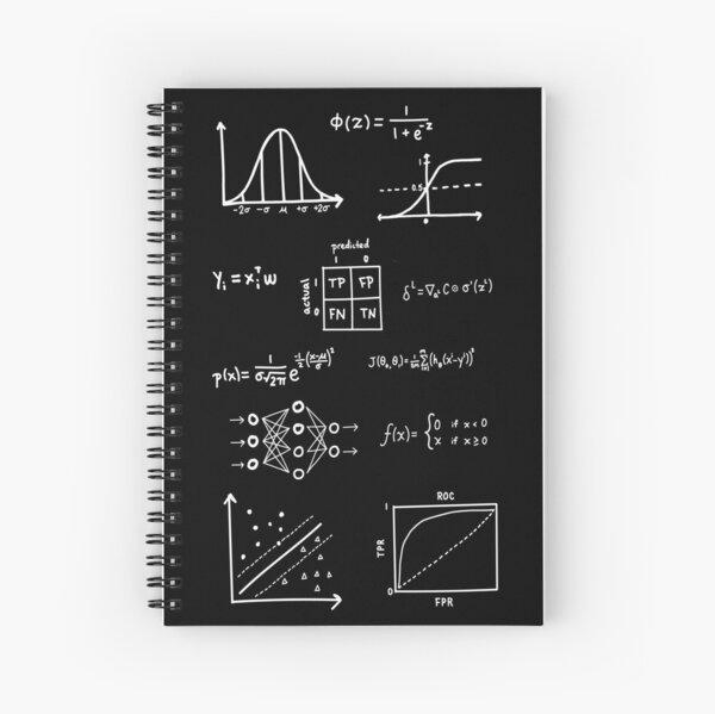
Example: Linear regression training or a gradient-based optimisation method.



Model

The learned system that maps input features to a prediction.

Example: A trained model that estimates chip power from voltage, frequency, and temperature.



Learned parameters

The internal numerical values adjusted during training to fit the examples.

Example: The coefficients that determine how strongly voltage and frequency influence a power prediction.

An easy way to remember the difference:

- The algorithm describes how learning happens.
- The parameters are the values learned.
- The model uses those values to make predictions.

3. Training vs. Inference

Aspect	**Training**	**Inference**
Purpose	Learn patterns from examples	Make predictions using learned patterns
Data	Usually training examples with target answers for supervised learning	New input data
Parameters	Adjusted during learning	Normally kept fixed
Error comparison	Loss is calculated to guide learning	Prediction can be compared with the true answer later
Output	A trained model	A prediction or estimate

Example: During training, a model learns from historical chip measurements and their actual power values. During inference, the trained model receives measurements from a new chip design and estimates its power consumption.

Inference does not ordinarily require the correct answer in advance.

4. How Does a Model Measure Its Errors?

A model makes predictions, but those predictions may not match the correct answers. A loss function measures how far the predictions are from the targets according to a chosen error measure.

For example:

Actual power	Predicted power	Absolute error
30 W	32 W	2 W
40 W	37 W	3 W
50 W	54 W	4 W

The absolute error is the magnitude of the difference between the actual and predicted values.

For these three examples, the mean absolute error (MAE) is:

$$\text{MAE}=\frac{2+3+4}{3}=3\text{ W}$$

This means the predictions differ from the actual measurements by an average of 3 W.

Common error measures include:

- Mean absolute error (MAE): Average magnitude of prediction errors.
- Mean squared error (MSE): Average squared prediction errors, which penalises larger errors more heavily.
- Classification loss: Measures how poorly predicted class probabilities match the correct labels.

A loss function provides a learning signal; the appropriate choice depends on the task.

5. What Is Optimisation?

Optimisation is the process of adjusting the model's parameters to improve its predictions according to the selected loss function.

The basic process is:

1. Make predictions using the current parameters.
2. Calculate the loss by comparing predictions with target answers.
3. Adjust the parameters using an optimisation method.
4. Repeat until training stops according to a chosen criterion.

For example, if a model consistently predicts too much power for a particular type of chip, training may adjust its parameters to reduce that error.

Different ML methods learn in different ways. Some use gradient-based optimisation, while others use methods such as tree splitting. Not every ML algorithm uses the same loss function or parameter-update procedure.

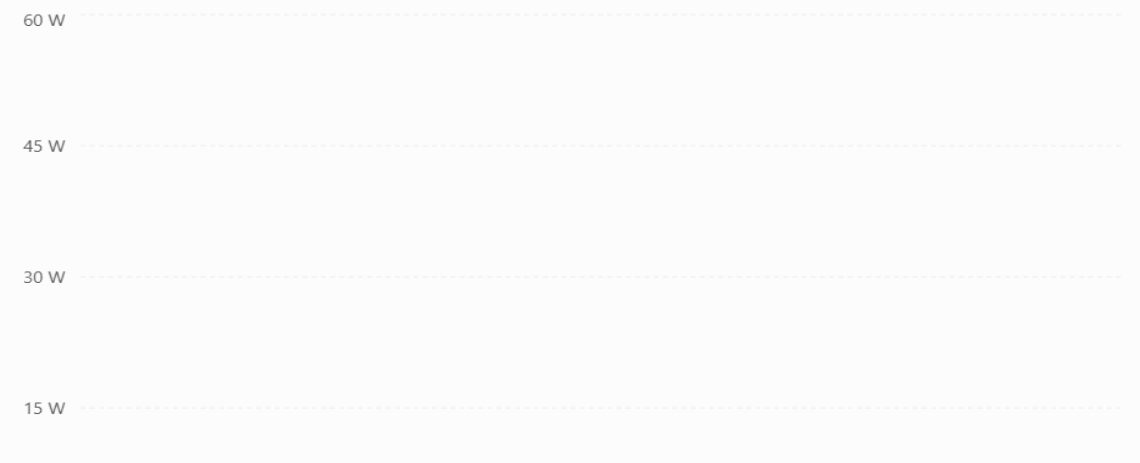
6. Understanding Linear Regression Conceptually

Linear regression is a simple ML method that learns a relationship between input features and a numerical outcome.

Suppose we want to predict a chip's power consumption using its operating frequency.

Frequency vs. power consumption

Example measurements used to learn a numerical relationship.



Training examples

The model receives frequency measurements and their corresponding power values. It learns a line that describes the relationship between the two variables.

Learned relationship

For this simplified example, the pattern is:

$$\text{Power} = 10(\text{Frequency}) + 10$$

Here, frequency is measured in GHz and power in watts.

Inference for a new input

If the new design operates at 2.5 GHz, the model predicts:

$$\text{Power} = 10(2.5) + 10 = 35\ \text{W}$$

The model estimates 35 W without needing the actual power measurement at prediction time.

Real chip power depends on many factors, including voltage, capacitance, circuit activity, and temperature. This one-feature example is only intended to illustrate how regression learns a relationship.

Conclusion

An ML model is built through a process of learning from examples. The algorithm determines how learning proceeds, the parameters are adjusted during training, and the trained model uses those parameters to make predictions during inference. Loss measures help quantify errors, while optimisation helps improve the model.

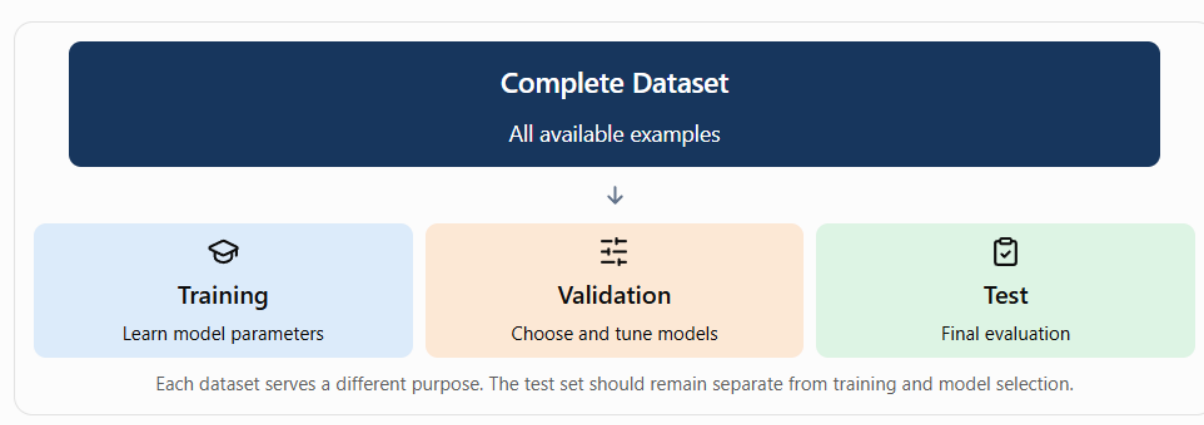
MISSION 6: Understand Evaluation and Generalisation

Research Question

How do we know whether a machine learning model has learned something useful?

A machine learning model is useful when it learns meaningful patterns from training data and can apply those patterns to new, unseen data. We evaluate a model using separate datasets and suitable performance metrics to check whether its predictions are reliable.

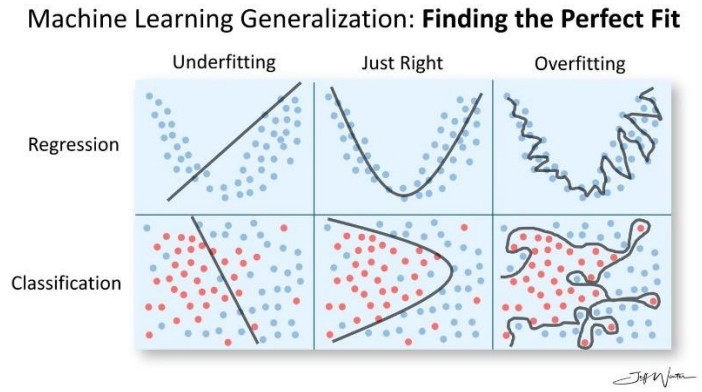
1. Training, Validation, and Test Datasets



Dataset	Purpose	Example
Training set	Teaches the model patterns from examples	Learn from chip measurements with known Pass/Fail results
Validation set	Helps select models, tune settings, and decide when to stop training	Compare different models before choosing one
Test set	Estimates final performance on unseen data	Evaluate the chosen model on previously unused chip designs

A common approach is to split the available data into 70% training, 15% validation, and 15% testing, although the best split depends on dataset size and the problem.

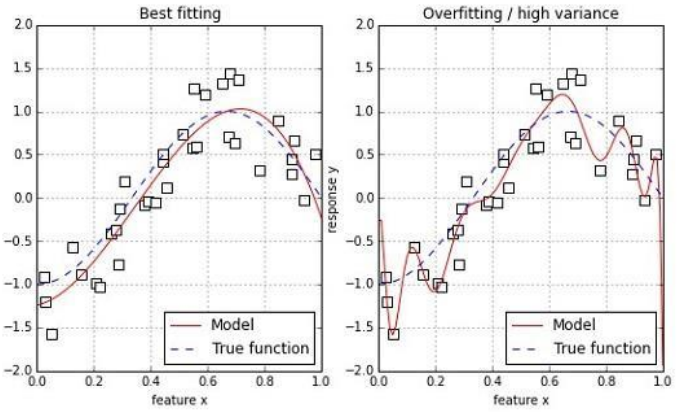
2. Generalisation, Overfitting, and Underfitting



Generalisation

The ability of a model to perform well on new data that it did not use during training.

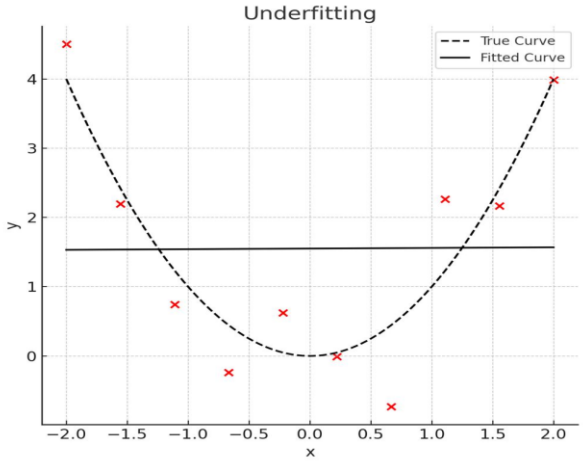
Example: A model accurately predicts timing success for new chip designs.



Overfitting

The model learns the training examples too closely, including noise or accidental patterns. It performs well on training data but poorly on unseen data.

Solution: Use regularisation, more representative data, simpler models, or appropriate early stopping.



Underfitting

The model is too simple or insufficiently trained to capture the important patterns. Its performance is poor on both training and unseen data.

Solution: Use a more suitable model, improve features, or train more effectively.

Training performance vs. unseen-data performance

Situation	Training performance	Test performance
Good generalisation	Good	Good
Overfitting	Very good	Poor or much worse
Underfitting	Poor	Poor

The goal is not simply to achieve the lowest training error. It is to build a model that performs well on data it has never seen.

3. Important Evaluation Metrics

Different metrics answer different questions. The appropriate metric depends on the task and the cost of making mistakes.

Classification metrics

Consider a model that predicts whether semiconductor chips will fail a quality test.

- True positive (TP): A faulty chip is correctly predicted as faulty.
- True negative (TN): A good chip is correctly predicted as good.
- False positive (FP): A good chip is incorrectly predicted as faulty.
- False negative (FN): A faulty chip is incorrectly predicted as good.

• **Metric**	**What question does it answer?**
Accuracy	What proportion of all predictions are correct?
Precision	Of all predicted positives, how many are actually positive?
Recall	Of all actual positives, how many did the model detect?
False positives	How many good cases were incorrectly flagged?
False negatives	How many real positive cases were missed?

Accuracy measures overall correctness. Precision measures the reliability of positive predictions. Recall measures how effectively the model finds positive cases.

Regression metric: Absolute Error

For regression, a prediction is usually a number rather than a category.

Absolute error is the size of the difference between an actual value and a predicted value.

For example, if actual chip power is 50 W and predicted power is 46 W:

$$\text{Absolute error} = |50 - 46| = 4 \text{ W}$$

Mean absolute error (MAE) averages the absolute errors across multiple examples. Lower MAE means predictions are closer to actual values on average, in the same units as the target.

4. Worked Example: Why Accuracy Can Be Misleading

Imagine testing a model on 100 chips:

- 95 chips are good.
- 5 chips are faulty.
- The model predicts that every chip is good.

Confusion matrix

Model predictions vs. actual chip condition

Actual condition	Predicted good	Predicted faulty
Good (95)	95 True negatives	0 false positives
Faulty (5)	5 False negatives	0 true positives

Accuracy	Recall for faulty chips
95%	0%
Looks impressive	Detects no faulty chips

Why is this model not useful?

The model gets 95 out of 100 predictions correct, so its accuracy is 95%. However, it fails to identify even one faulty chip.

This happens because the dataset is imbalanced: good chips greatly outnumber faulty chips. The model can achieve high accuracy simply by always predicting the majority class.

What additional metrics would help?

- Recall: In this example, recall is 0%. It reveals that all faulty chips are missed.
- Precision: Here it is undefined because the model predicts no faulty chips. In practice, software may report zero by convention.
- F1-score: Combines precision and recall into a single measure. It is useful when both are important, although it does not replace examining them separately.
- Confusion matrix: Shows the actual counts of correct and incorrect predictions, helping reveal which mistakes are being made.

For semiconductor quality control, recall may be especially important when failing to detect a faulty chip is costly. However, false positives can also waste testing resources, so precision and the practical costs of both types of errors should be considered.

5. Why the Metric Must Match the Problem

Problem	Useful metric	Why?
Detecting faulty chips	Recall, precision, F1-score	Missing faults and incorrectly flagging good chips have different costs
Spam detection	Precision and recall	False spam flags and missed spam have different consequences
Predicting chip power	MAE or RMSE	Measures the size of numerical prediction errors
Balanced classification	Accuracy, alongside other metrics	Overall correctness may be informative when classes are reasonably balanced

No single metric is best for every ML problem. The evaluation should reflect the purpose of the model, the distribution of the data, and the real-world consequences of errors.

Conclusion

Evaluation helps determine whether a model has learned useful patterns rather than merely memorising training examples. Training data teaches the model, validation data supports model selection, and test data estimates performance on unseen examples. Metrics such as accuracy, precision, recall, and absolute error provide different perspectives on performance.

MISSION 7: Understand Why Machine Learning Systems Fail

Research Question

Even if a model works during testing, why might it fail in reality?

A machine learning (ML) model can perform well during testing but fail in real-world situations because the data may be poor, biased, incomplete, or different from the data used during training. Other causes include data leakage, changing real-world conditions, and incorrect predictions that are not properly reviewed by humans.

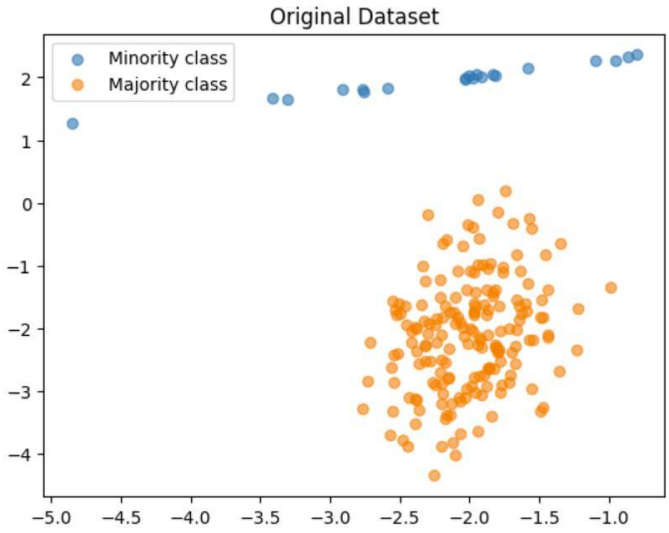
1. Common Reasons Why ML Systems Fail



1. Poor-quality data

Missing values, incorrect measurements, duplicate records, noisy observations, and inconsistent labels can prevent a model from learning reliable patterns.

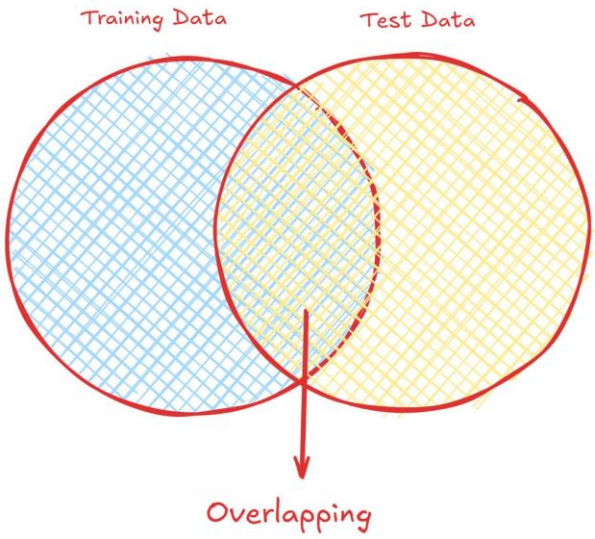
Example: A chip marked Pass even though its measured timing violates the required limit.



2. Unrepresentative data and bias

Training data may not represent all the situations the model will encounter. The model can perform well on common cases but poorly on underrepresented groups or conditions.

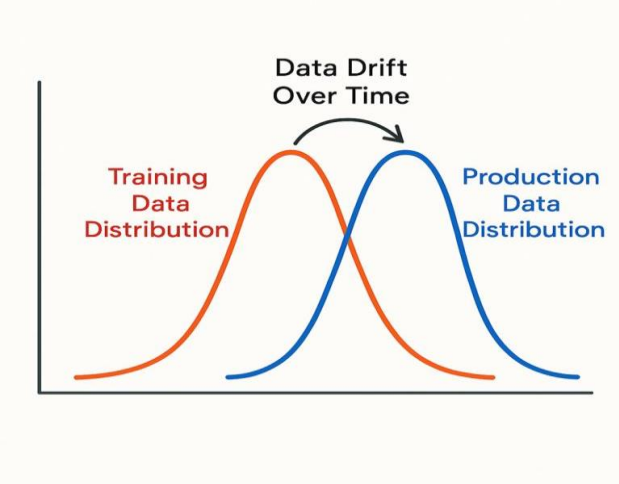
Example: A chip-failure model trained mostly on low-temperature tests performs poorly at high temperatures.



3. Data leakage

Information unavailable at prediction time accidentally enters training or evaluation, making performance appear better than it really is.

Example: A model predicting manufacturing failure uses a test result recorded only after the failure occurs.



4. Changing real-world data

Real-world conditions can change after deployment. This is called data drift when input distributions change, or concept drift when the relationship between inputs and outcomes changes.

Example: A model trained on an older chip manufacturing process becomes less accurate after the process is updated.



5. Insufficient human oversight

Incorrect predictions can cause harm when users trust them without verification, especially in high-impact applications.

Example: A quality-control system approves a faulty chip without further inspection.

2. Understanding Data Bias and Performance Differences

Data bias occurs when the examples used to train a model systematically misrepresent the real-world population or conditions.

For example, suppose a semiconductor failure model is trained using:

- 90% low-temperature measurements.
- 10% high-temperature measurements.

If high-temperature failures are more common and the model has too few examples to learn this pattern, it may miss important faults under high-temperature conditions.

Performance may also differ across datasets because they contain different distributions, measurement procedures, manufacturing processes, or proportions of rare cases.

Important distinction: A performance difference is evidence that needs investigation; it does not automatically prove that bias is the cause.

3. Why Data Leakage Makes Evaluation Misleading

Data leakage happens when information that should be unavailable during training or evaluation gives the model an unfair advantage.

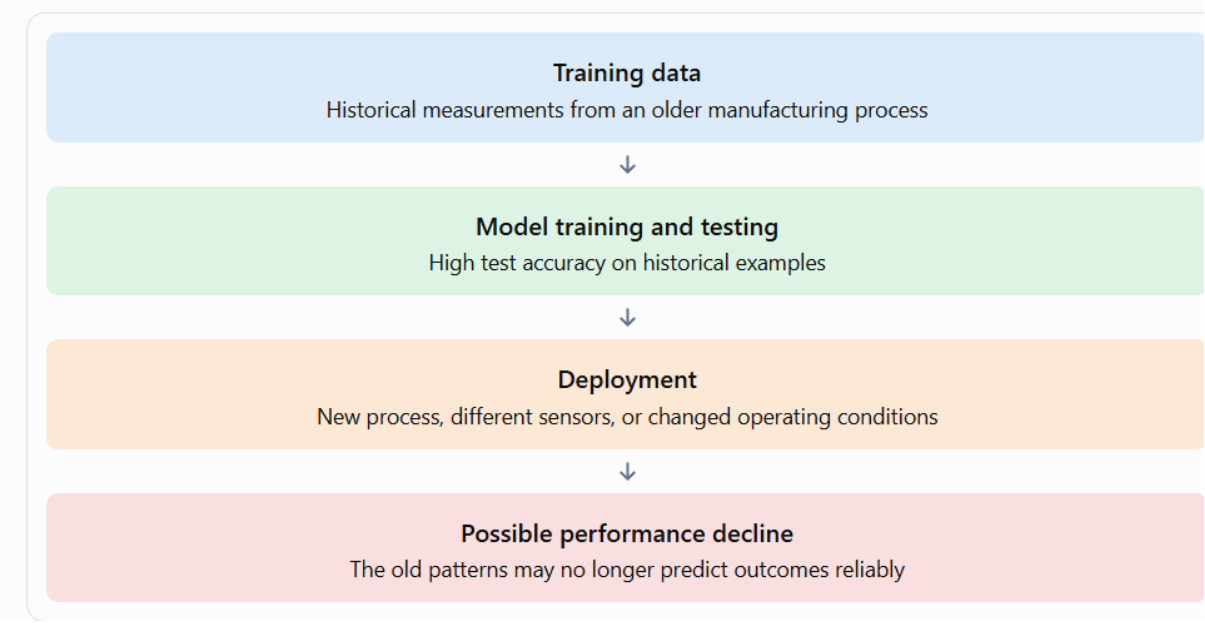
Common examples include:

- Target leakage: A feature contains information that directly reveals the answer.
- Train-test contamination: Test examples or their information influence model training or selection.
- Temporal leakage: Future information is used to predict an earlier event.
- Preprocessing leakage: Data transformations, such as feature scaling or feature selection, are fitted using the entire dataset before splitting it.

For example, if measurements from the same chip design appear in both training and test sets, the model may appear to generalise well while effectively recognising previously seen designs.

To reduce leakage, split data appropriately before fitting preprocessing steps, keep the test set untouched during model selection, and use time-based or group-based splits when the real deployment situation requires them.

4. How Real-World Changes Reduce Performance



A model that worked previously may need recalibration, retraining, improved features, or a different modelling approach. Before making changes, teams should confirm that performance has genuinely declined and investigate the underlying cause.

5. Deliverable: Analysis of Three Hypothetical ML Failures

The following scenarios are illustrative examples.

Failure 1: Faulty chips are incorrectly approved

Classification failure

Situation: A model reports 98% accuracy during testing but misses many faulty chips after deployment.

Likely cause: The test dataset contains very few faulty chips, so high accuracy hides poor recall. Labels may also be inconsistent.

Evidence to confirm it:

- Confusion matrix and recall for faulty chips.
- Number of faulty examples in training, validation, and test sets.
- Audit of actual failure labels and test conditions.

Possible mitigation:

- Evaluate recall, precision, and false-negative rates.
- Collect representative examples of faulty chips.
- Review label quality and use additional inspection for uncertain cases.

Failure 2: A chip-power model performs poorly on a new process

Data drift

Situation: A regression model accurately predicts power on older chip designs but makes large errors on newly manufactured designs.

Likely cause: The manufacturing process, operating conditions, or relationship between input measurements and power consumption has changed.

Evidence to confirm it:

- Compare input distributions between historical and current data.
- Measure MAE or RMSE on recently verified chip measurements.
- Examine errors by manufacturing process, temperature, and frequency.

Possible mitigation:

- Collect verified measurements from the new process.
- Retrain or recalibrate the model when appropriate.
- Monitor prediction errors and establish a fallback procedure when reliability is insufficient.

Failure 3: A model appears excellent but fails after launch

Data leakage

Situation: A model achieves almost perfect test results, but its real-world predictions are much worse.

Likely cause: A feature accidentally contains the final inspection result, or related records from the same chip or production batch occur in both training and test data.

Evidence to confirm it:

- Review every feature and when it becomes available.
- Check for overlapping chip IDs, batches, or near-duplicate records.
- Repeat evaluation using a clean, group-based or time-based split.

Possible mitigation:

- Remove features unavailable at prediction time.
- Separate related records across the data splits.
- Refit preprocessing and feature selection using training data only, then evaluate once on an untouched test set.

6. The Importance of Human Oversight

Human oversight helps detect errors that automated metrics may not reveal. It is especially important when incorrect predictions could cause safety issues, expensive production mistakes, or significant business losses.

Useful safeguards include:

- Reviewing uncertain or high-risk predictions.
- Setting thresholds based on the costs of false positives and false negatives.
- Monitoring performance after deployment.
- Keeping an audit trail of predictions, outcomes, and model changes.
- Providing a way to override or disable the model when it is unreliable.

Human oversight does not guarantee that every error will be caught, so it should complement strong data practices, suitable evaluation, and continuous monitoring.

Conclusion

ML systems can fail because of poor-quality data, unrepresentative examples, data leakage, changing real-world conditions, and insufficient oversight. Good testing is necessary, but it is not enough: evaluation must reflect the conditions in which the model will actually be used.

MISSION 8: Does This Problem Actually Need Machine Learning?

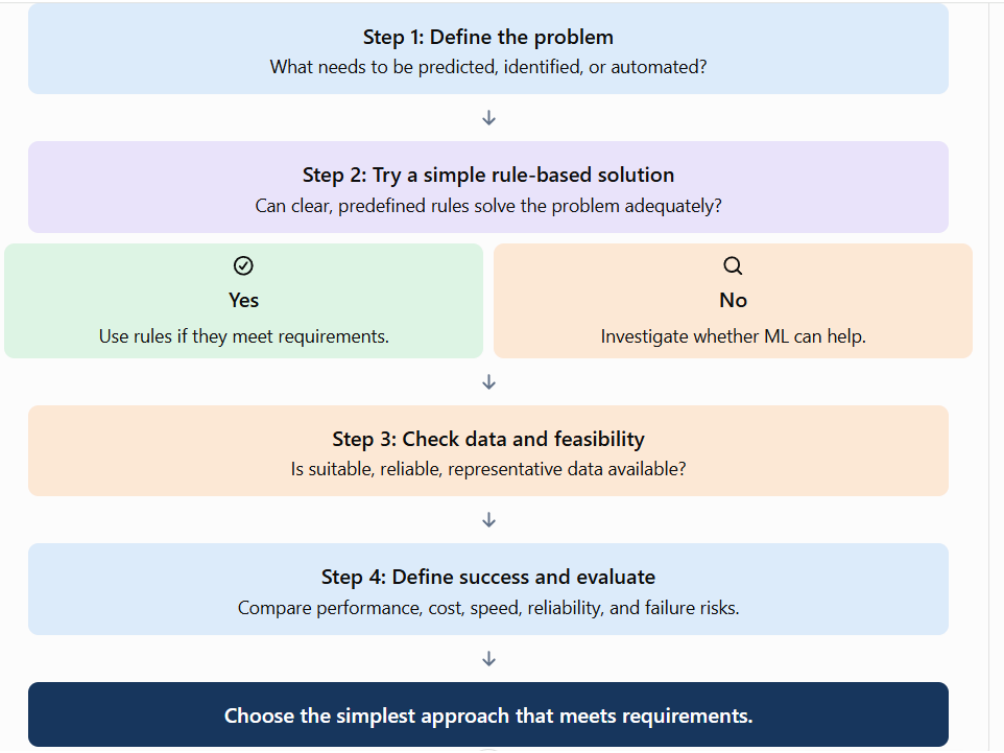
Research Question

When should an engineer use ML instead of a fixed rule or conventional program?

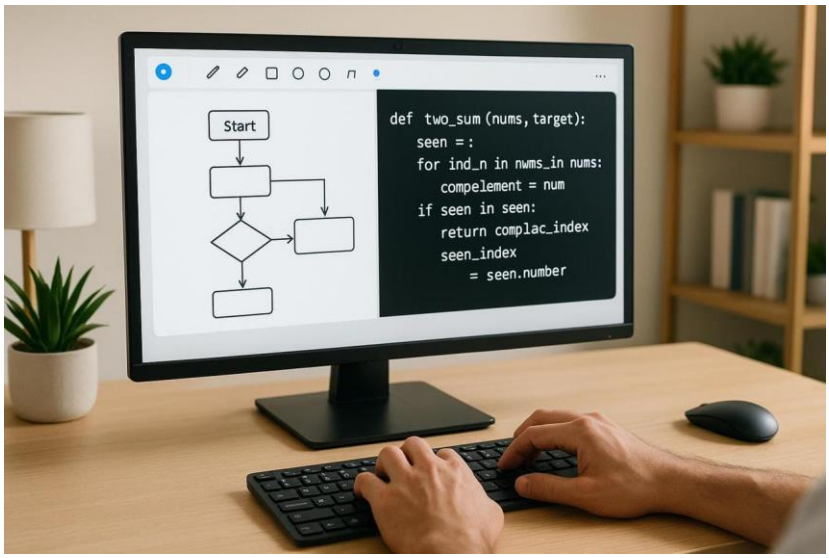
Machine learning is useful when a system needs to learn patterns from data that are difficult to describe with fixed rules. However, if a simple rule-based program solves the problem accurately, reliably, and efficiently, ML may be unnecessary.

The best approach starts with understanding the problem, defining the desired outcome, checking available data, and deciding how success will be measured.

1. How to Decide Whether ML Is Necessary



2. Rule-Based Programming vs. Machine Learning



Rule-Based Programming

Developers explicitly write the instructions that determine what the system does.

- Works well when rules are clear and stable.
- Does not require training data.
- Usually easier to explain and test.
- Can become difficult to maintain when there are many exceptions.



Machine Learning

A model learns patterns from examples and uses them to predict or identify outcomes.

- Useful when relationships are complex or difficult to express as rules.
- Requires suitable data and evaluation.
- Can adapt through retraining when patterns change.
- May produce incorrect or difficult-to-explain predictions.

3. Deliverable: Compare Both Approaches to the Same Problem

Example: Detecting faulty semiconductor chips

Suppose an engineering team wants to identify chips that fail quality checks before shipping.

Option A: Fixed rules

Example rule

If measured delay exceeds 10 ns, flag the chip for timing failure.

Advantages

- Simple to implement and explain.
- No training dataset required.
- Predictable and easy to audit.

Trade-offs and failure modes

- May miss faults caused by combinations of several measurements.
- Thresholds may need manual updates when manufacturing conditions change.
- A single threshold may not work for every chip type.

Option B: Machine learning

Example model

Train a classifier using historical measurements such as delay, power, voltage, temperature, and verified Pass/Fail results.

Advantages

- Can learn complex relationships between multiple measurements.
- Can identify patterns that are difficult to describe with individual rules.
- Can be retrained using new, verified examples.

Trade-offs and failure modes

- Needs representative data and reliable labels.
- May overfit historical measurements or miss rare faults.
- Requires monitoring, maintenance, and suitable computing resources.
- Incorrect predictions can lead to faulty chips being approved or good chips being rejected.

4. Data Requirements and Trade-Offs

Consideration	Rule-based solution	ML solution
Data requirements	May need only current measurements and specifications	Usually needs suitable training examples; supervised ML needs labels
Initial development	Often simpler for clear rules	Requires model development and evaluation
Complex patterns	Limited by the rules developers specify	Can learn complex relationships from examples
Explainability	Usually straightforward	Depends on the model; some are difficult to interpret
Changing conditions	Rules need manual revision	May need monitoring, recalibration, or retraining
Failure risks	Missing exceptions or outdated rules	Bias, poor generalisation, leakage, and prediction errors
Best suited for	Stable, clearly defined logic	Complex patterns that can be learned from representative data

Neither approach is automatically better. A hybrid solution may be most effective: explicit rules can enforce mandatory safety limits while an ML model identifies more subtle potential faults.

5. How Should Success Be Measured?

Before selecting a technique, define measurable criteria.

For the chip-quality example, useful measures include:

- Recall: What proportion of faulty chips does the system detect?
- Precision: What proportion of chips flagged as faulty are actually faulty?
- False-negative rate: What proportion of faulty chips are missed?
- Latency: How quickly does the system produce a result?
- Cost: What are the development, computing, testing, and maintenance costs?

An ML solution should be compared with a sensible baseline, such as the existing rule-based system, using representative data that has not been used to train or tune the model.

If ML does not provide a meaningful improvement in the outcomes that matter, the simpler solution may be preferable.

6. What If an ML Prediction Is Incorrect?

An engineer should consider the consequences before deploying a model.

For example, if a model incorrectly approves a faulty chip, the product could fail in the field. If it incorrectly rejects a good chip, production costs may increase.

Possible safeguards include:

- Keep mandatory engineering limits as explicit rules.
- Send uncertain or high-risk predictions for human review.
- Monitor real-world errors and changes in data.
- Provide a safe fallback when the model is unavailable or unreliable.
- Retrain or replace the model only after careful evaluation.

Conclusion

The decision to use machine learning should depend on the problem, not on the popularity of the technology. Rule-based programming is often best when logic is clear and stable. ML is worth considering when complex patterns cannot be captured adequately by fixed rules and suitable data is available.

MISSION 9: Connect the Full ML Map to Semiconductor Engineering

Case Study: Machine Learning for Early PPA Estimation in VLSI Design

Research question: Where does machine learning fit into the semiconductor value chain?

1. Introduction

Power, Performance, and Area (PPA) are three important measures of semiconductor design quality. Engineers aim to minimize power consumption and chip area while meeting performance requirements such as operating frequency and timing constraints.

Traditionally, evaluating a Register Transfer Level (RTL) design involves running Electronic Design Automation (EDA) tools through synthesis and, when necessary, physical implementation. These processes can be computationally expensive and slow down early design exploration.

Machine learning offers a possible solution: learn from previous designs and their measured results to estimate PPA earlier in the design process. Engineers can then use these estimates to identify promising designs before investing in more time-consuming implementation steps.

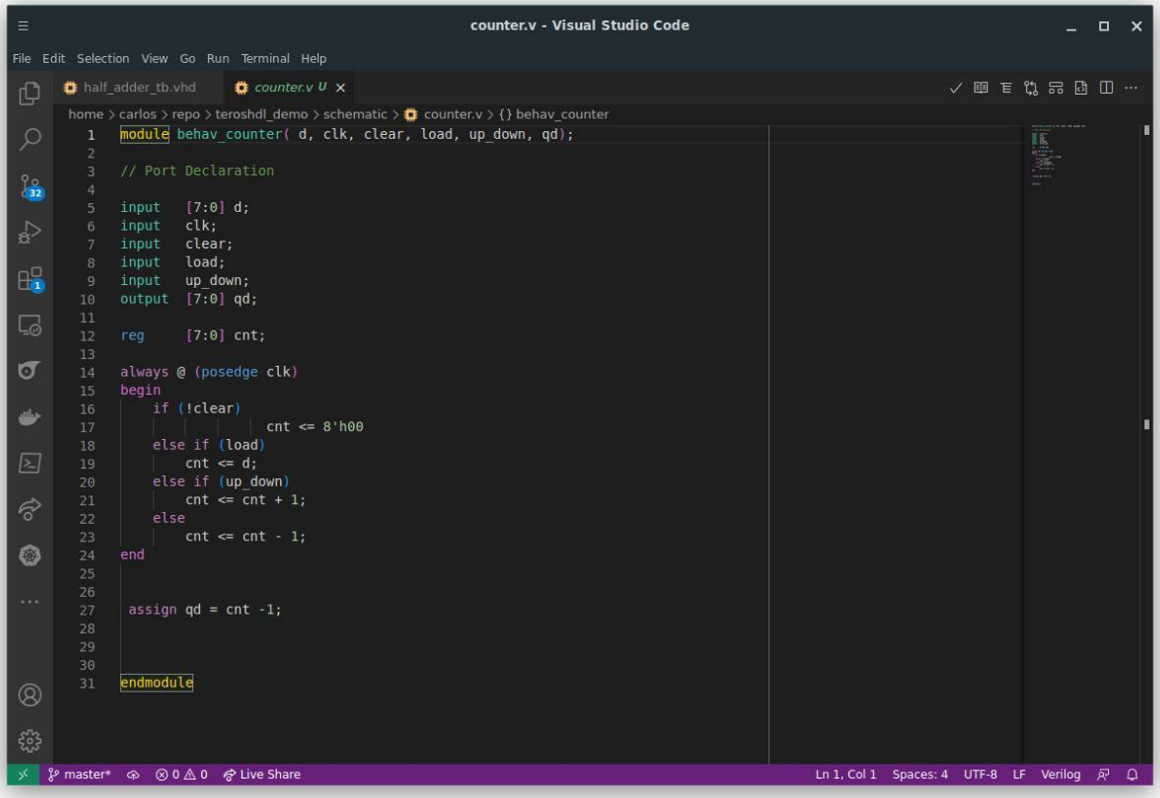
This case study focuses on early PPA estimation from RTL code, an application of ML in the semiconductor design value chain.

2. Engineering Problem and Required Data

Engineering problem: How can engineers estimate a chip design's power consumption, timing performance, and area before completing the full implementation flow?

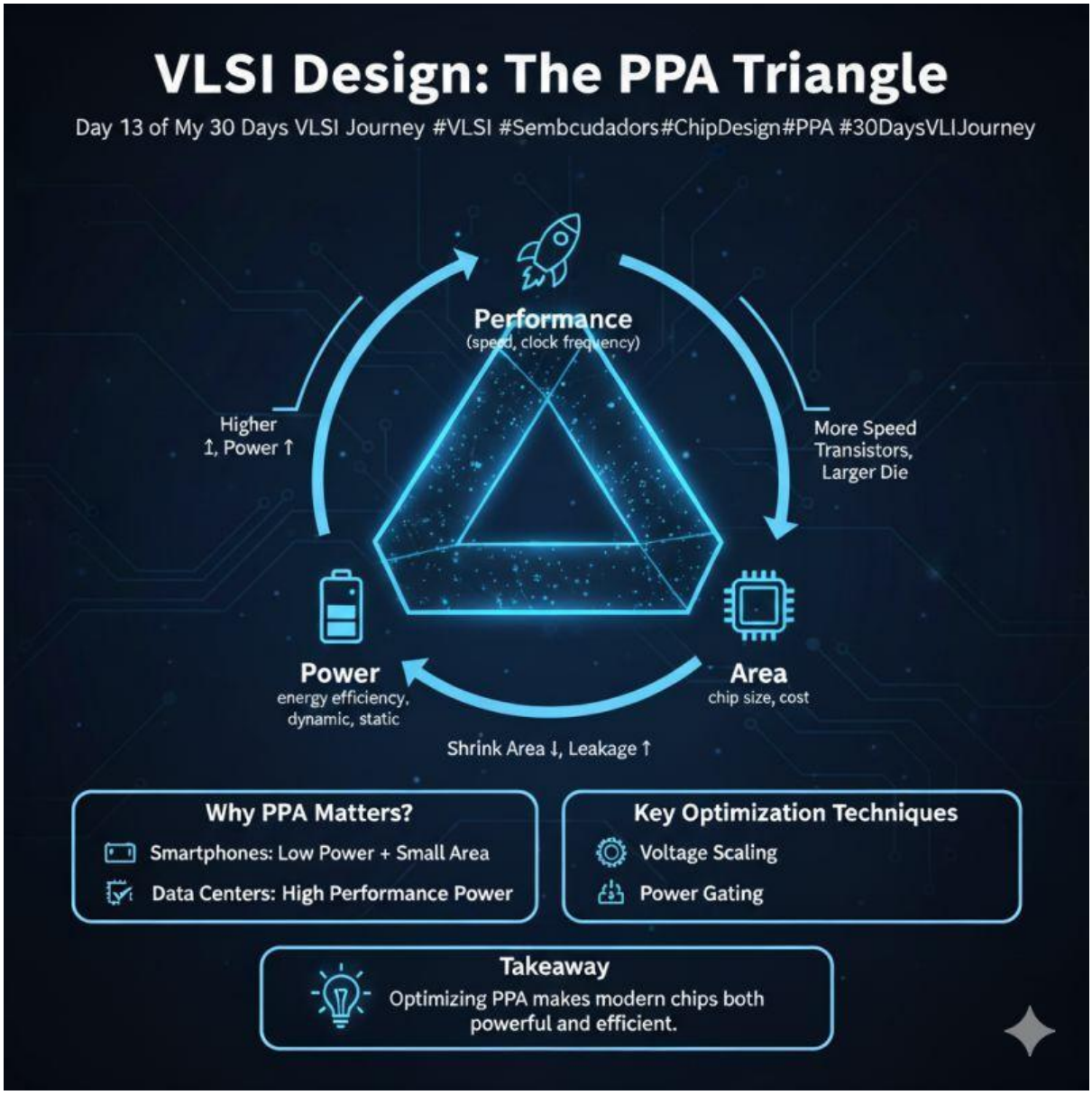
The model would learn from historical RTL designs paired with results produced by trusted synthesis or implementation tools.

Data required for the model



Input data (features)

- RTL source code or a structured representation of its logic.
- Operator types, bit widths, logic depth, and circuit complexity.
- Target technology library and relevant design constraints.
- Where available, synthesis settings and operating conditions.



Target data (known outcomes)

- Power consumption.
- Area, such as standard-cell area.
- Timing metrics, such as worst negative slack (WNS) and total negative slack (TNS).

These targets are obtained from reference EDA runs under documented conditions.

The dataset must contain reliable results from sufficiently varied designs. The model's predictions will be less trustworthy if the training examples do not represent the circuit types, technology libraries, or operating conditions encountered in practice.

3. Intended Model Output, Task Type, and Learning Approach

The main task is regression, because the model estimates numerical PPA metrics rather than only assigning a category.

Component	Description
Task type	Regression
Learning approach	Supervised learning
Input	RTL-derived features and relevant design constraints
Output	Estimated power, area, WNS, or TNS
Training signal	Reference values from EDA tool reports
Intended use	Early design screening and prioritisation

Separate models may be trained for power, timing, and area because these quantities depend on different design characteristics.

4. Evidence from Research

Two primary research studies illustrate how this application is being investigated.

5. How Should the Model Be Evaluated?

An engineer must verify that the model is useful beyond the examples used for training.

Recommended evaluation methods include:

- Mean absolute error (MAE): Measures the average size of prediction errors in the target's units.
- Root mean squared error (RMSE): Gives larger errors greater weight.
- Correlation: Measures how closely predictions track actual results, but high correlation alone does not guarantee small prediction errors.
- Held-out design testing: Tests on RTL designs excluded from training and model selection.
- Cross-technology evaluation: Checks whether performance remains acceptable across technology libraries, process corners, and design types.

6. Limitations and Engineering Considerations

Several limitations must be investigated before using ML-based PPA estimation in practice.

1. Limited training data: High-quality RTL designs paired with trustworthy reference results may be expensive to generate.
2. Generalisation: A model trained on one set of circuits may perform poorly on unfamiliar architectures or technology nodes.
3. Approximation error: Early estimates are made before all synthesis and physical-design effects are known.

4. Changing conditions: Different constraints, libraries, tools, and implementation settings can change the true PPA results.
5. False confidence: Engineers may reject a good design or select a poor one if they treat estimates as exact measurements.

Conclusion

Early PPA estimation is a useful example of how ML fits into the semiconductor value chain. By learning from RTL representations and reference EDA results, a supervised regression model can provide early estimates of power, performance, and area. Research such as Master RTL demonstrates the potential to improve design exploration, while also highlighting the challenge of generalising across different designs.